

Dispense di Algoritmi e Complessità

Marco Galletti

A.A. 2024/2025

Sommario

Questa dispensa contiene un riassunto dei principali concetti trattati nel corso di Algoritmi e Complessità con i professori Alessandro Panconesi (complessità) e Flavio Chierichetti (algoritmi) del corso di laurea di "Scienze matematiche per l'intelligenza artificiale" presso l'Università Roma "La Sapienza".

Indice

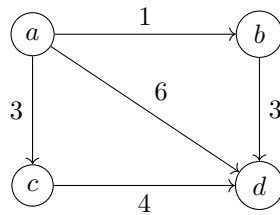
1	Introduzione	1
1.1	Algoritmo di Stable Matching	3
1.2	Notazioni asintotiche	6
2	Algoritmi Greedy	9
2.1	Interval Scheduling	9
2.2	Interval Partitioning	10
2.3	Compressione dei Dati	12
2.4	Prefix-Code e alberi binari	14
3	Programmazione Dinamica	21
3.1	Weighted Interval Scheduling	21
4	Machine Learning	23
4.1	Algoritmi non-random per il problema degli esperti	26
4.2	Clustering	28
5	Fondamenti di complessità	29
6	Macchina di Turing	40
7	Problemi indecidibili e non	43

1 Introduzione

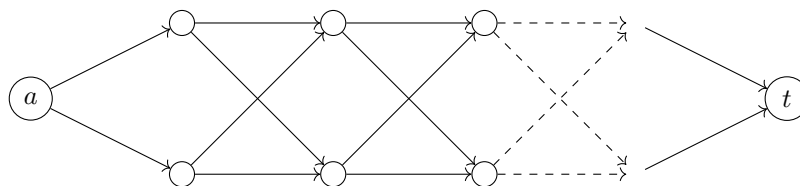
L'algoritmica è la disciplina che studia le procedure per risolvere classi di problemi in maniera deterministica e non ambigua. Queste procedure sono dette algoritmi e sono composte da una sequenza finita di passi elementari che trasformano l'input del problema in un output.

Osservazione 1.1. Ogni classe di problemi può avere più di una soluzione (algoritmo) che la risolve.

Esempio 1.2 (Cammino minimo in un grado diretto pesato). Dato il seguente grafo diretto pesato:



Si trovi il percorso minimo da a a d . Idealmente si potrebbe pensare di proporre un algoritmo che provi tutti i possibili percorsi da a a d e restituisca quello con il peso minore. Questa soluzione trova effettivamente il cammino minimo, ma non è efficiente dato che il numero di percorsi possibili cresce esponenzialmente con la grandezza del grafo. Infatti seppure per il grafo sopra il numero di percorsi è molto limitato e visualizzabile, per grafi più grandi il numero di percorsi diventa non indifferente. Se consideriamo il grafo qui sotto:



L'approccio banale risulta molto inefficace. Nello specifico si dice che ha crescita esponenziale di ordine $O(2^n)$. Nel corso di Tecniche di Programmazione si sono affrontate sia le basi della teoria della complessità computazionale, sia l'analisi di un algoritmo ottimo per la risoluzione del problema del cammino minimo (i.e. Dijkstra per grafi con pesi positivi).

Problema 1.1. Si trovi il valore massimo in un vettore $\mathbf{v}$ con n elementi.

Soluzione 1.1.1. Di seguito l'analisi della complessità computazionale dell'algoritmo:

```

1: function FINDMAX( $\mathbf{v}, n$ )
2:    $max \leftarrow -\infty$ 
3:   for  $i \leftarrow 1$  to  $n$  do
4:     if  $\mathbf{v}[i] > max$  then
5:        $max \leftarrow \mathbf{v}[i]$ 
6:     end if
7:   end for
8:   return  $max$ 
9: end function

```

- Il ciclo **for** viene eseguito n volte.
- All'interno del ciclo vengono eseguite $O(1)$ operazioni.

Quindi la complessità computazionale dell'algoritmo è $O(n)$.

Problema 1.2. Si determini se l'intero x è nel vettore $\mathbf{v}$ con n elementi.

Soluzione 1.2.1.

Si intuisce che i due algoritmi presentati presentano una struttura molto simile, infatti sono detti **algoritmi di scansione lineare**.

Problema 1.3. Dato un vettore $\mathbf{v}$ con n elementi contenente tutti gli interi da 0 a $n - 1$ con l'esclusione di uno di questi. Si determini quale intero manca. A questo problema presentiamo due soluzioni: una naive e una più efficiente.

Soluzione 1.3.1. Questa soluzione ha complessità computazionale $O(n^2)$ (per ogni elemento del vettore che ha dimensionalità n si esegue una scansione lineare che ha complessità $O(n)$).

Soluzione 1.3.2. Questa soluzione ha complessità computazionale $O(n)$.

```

1: function FIND( $\mathbf{v}, x, n$ )
2:   for  $i \leftarrow 1$  to  $n$  do
3:     if  $\mathbf{v}[i] = x$  then
4:       return true
5:     end if
6:   end for
7:   return false
8: end function

```

Algorithm 1 Soluzione Naive

```

1: function FINDMISSING( $\mathbf{v}, n$ )
2:   for  $i \leftarrow 1$  to  $n$  do
3:     if not FIND( $\mathbf{v}, i, n$ ) then
4:       return  $i$ 
5:     end if
6:   end for
7: end function

```

Dimostrazione. Verifichiamo di seguito la correttezza dell'algoritmo

$$S = \sum_{x \in V} x$$

$$T = \sum_{i=0}^{n-1} i$$

Se M è l'intero mancante, allora si ha che $S = \underbrace{\left(\sum_{i=0}^n i \right)}_T - M$

$$\implies M = T - S = \sum_{i=0}^{n-1} i - \sum_{x \in V} x$$

□

1.1 Algoritmo di Stable Matching

L'algoritmo di stable matching è un algoritmo che risolve il problema del matching stabile. Questo problema consiste nel trovare un matching stabile tra due insiemi di elementi, dove ad ogni elemento di un insieme è associato un ordine di preferenza per gli elementi dell'altro insieme. Si considerino due insiemi di elementi A e B (disgiunti $A \cap B = \emptyset$) con $|A| = |B| = n$. Vogliamo creare delle coppie $\{a, b\}$ con $a \in A$ e $b \in B$ tali che le preferenze siano rispettate.

Algorithm 2 Soluzione Efficiente

```

1: function FINDMISSING( $\mathbf{v}, n$ )
2:    $sum \leftarrow 0$ 
3:   for  $i \leftarrow 1$  to  $n$  do
4:      $sum \leftarrow sum + \mathbf{v}[i]$ 
5:   end for
6:    $T \leftarrow \frac{(n-1)n}{2}$ 
7:   return  $T - sum$ 
8: end function

```

Esempio 1.3. Siano $A = \{a_1, a_2\}$ e $B = \{b_1, b_2\}$ con le seguenti preferenze:

$$\begin{aligned} a_1 : b_2 > b_1 &\Rightarrow a_1 \text{ preferisce } b_2 \text{ a } b_1 \\ b_2 : a_1 > a_2 &\Rightarrow b_2 \text{ preferisce } a_1 \text{ a } a_2 \end{aligned}$$

Possiamo creare la coppia (a_1, b_2) .

Definizione 1.4 (Matching perfetto). Un matching perfetto è un matching in cui ogni elemento di A è associato ad un elemento di B e viceversa (ovviamente). Siano A e B tali che $|A| = |B| = n$ e $A \cap B = \emptyset$. Definiamo un matching perfetto come un accoppiamento degli elementi di A con gli elementi di B tale che:

$$\text{Sia } M \subseteq \{\{a, b\} \mid a \in A, b \in B\} = A \times B$$

M è un matching perfetto se:

$$\forall a \in A, \exists! b \in B \text{ tale che } \{a, b\} \in M$$

Quindi se corrisponde a una biiezione tra A e B . Segue ovviamente che $\forall b \in B, \exists! a \in A$ tale che $\{a, b\} \in M$.

Osservazione 1.5. Un matching perfetto è un caso particolare di matching stabile.

Definizione 1.6 (Matching stabile). Siano A e B insiemi tali che $|A| = |B| = n$ e $A \cap B = \emptyset$. Si supponga che ogni elemento di A abbia una lista di preferenze su B e viceversa (la lista deve essere di ordine totale). Sia M un matching perfetto da A a B . Si dice che due coppie distinte $\{a, b\}, \{a', b'\} \in M$ sono instabili se:

- a preferisce b' a b
- b' preferisce a ad a'

E quindi il matching non è stabile. Si dice che un matching perfetto M è stabile se, fissate liste di preferenze per ogni nodo, non esistono coppie instabili.

Di seguito presentiamo l'algoritmo di Gale-Shapley per il calcolo di un matching stabile.

Algorithm 3 Gale-Shapley

```

1: function GALESHAPLEY( $A, B$ )
2:    $M \leftarrow \emptyset$ 
3:   while  $\exists a \in A$  tale che  $a$  non è accoppiato do
4:     Sia  $a_i \in A$  un elemento libero di  $A$  che non è accoppiato
5:     Sia  $B' \subseteq B$  l'insieme degli elementi di  $B$  che  $a_i$  non ha ancora proposto  $\Rightarrow B' \neq \emptyset$ 
6:     Sia  $b_j \in B'$  l'elemento che è più in alto nella lista di preferenze di  $a_i$   $\Rightarrow a_i$  propone a  $b_j$ 
7:     if  $b_j$  è libero then
8:        $M \leftarrow M \cup \{a_i, b_j\} \Rightarrow a_i$  e  $b_j$  non sono più liberi
9:     else
10:      Sia  $a_k$  l'attuale partner di  $b_j$ 
11:      if  $b_j$  preferisce  $a_k$  (ovvero  $a_k > a_i$ ) then
12:         $b_j$  rifiuta e  $a_i$  rimane libero  $\Rightarrow$  resta  $\{a_k, b_j\}$ 
13:      else
14:         $M \leftarrow M \cup \{a_i, b_j\} \Rightarrow a_i$  e  $b_j$  non sono più liberi
15:         $a_k$  diventa libero  $\Rightarrow$  si distrugge  $\{a_k, b_j\}$  e si crea  $\{a_i, b_j\}$ 
16:      end if
17:    end if
18:  end while
19: end function

```

Lemma 1.7 (L1). Ogni elemento $b \in B$ rimane accoppiato dalla **prima volta** che riceve una richiesta, fino alla terminazione. Inoltre i partner di b **non peggiorano** nel tempo.

Dimostrazione. b accetta la prima proposta che riceve; se dovesse ricevere una proposta da a_i quando è già in coppia con a_k , la accetterà se e solo se a_i è più preferito di a_k , e queste miglioreranno il partner dal punto di vista di b . $\square$

Lemma 1.8 (L2). $\forall a \in A$ la sequenza di proposte fatte da a non migliora nel tempo.

Dimostrazione. Se a propone a b e viene rifiutato, allora a dovrà proporsi a un elemento che si trova più in basso nella sua lista di preferenze. $\square$

Teorema 1.9. L'algoritmo di Gale-Shapley termina dopo al più n^2 iterazioni del ciclo *while*.

Dimostrazione. Ogni $a_i \in A$ può proporsi ad al più $|B| = n$ elementi di B . In ogni iterazione, un qualche a_i si propone a un qualche b_j , cui non si era proposto in precedenza. Ma visto che ci sono n elementi in A (lavoriamo considerando $|A| = |B|$ e $A \cap B = \emptyset$), il numero di iterazioni è al più $|A| \cdot |B| = n^2$. $\square$

Il numero di iterazioni è quindi limitato. Inoltre, l'algoritmo termina con un matching stabile.

Lemma 1.10 (L3). Se $a \in A$ è libero ad un certo punto dell'algoritmo, allora esiste $b \in B$ tale che a non ha ancora proposto a b .

Dimostrazione. Per contraddizione supponiamo che ad un certo punto $a^* \in A$ sia libero e abbia già proposto a tutti gli elementi di B . Allora, per il lemma L1 1.7 ogni $b \in B$ rimane in una coppia dalla prima proposta che riceve fino alla fine dell'esecuzione. Inoltre, quando $a \in A$ si propone a $b \in B$ libero, b accetta. Quindi, perché a^* sia libero dopo n proposte, è necessario che al momento della sua ultima proposta (l' n -esima proposta) ogni $b \in B$ deve essere già in una coppia. Ma dato che l'algoritmo non permette **mai** ad un elemento di avere più di un partner (in ogni iterazione si distrugge una coppia e se ne crea una nuova), e visto che $|A| = |B|$, abbiamo una contraddizione. Quindi è impossibile che ogni $b \in B$ sia già in coppia se $a^* \in A$ è libero. $\square$

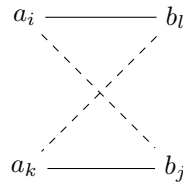
Lemma 1.11 (L4). L'algoritmo di Gale-Shapley termina con un matching perfetto.

Dimostrazione. In ogni momento l'insieme delle coppie di M forma un matching (non esiste nessun $a \in A$ e nessun $b \in B$ con almeno due partner). Per il lemma L3 1.10, alla fine dell'esecuzione non possono esistere elementi liberi in A . Visto che $|A| = |B| = n$, e che l'output dell'algoritmo è un matching, non possono neanche esistere elementi liberi in B . Il matching è quindi perfetto. $\square$

Teorema 1.12. L'algoritmo di Gale-Shapley termina con un matching stabile.

Dimostrazione. Per il lemma L4 1.11, il matching M che l'algoritmo restituisce è perfetto. Poniamo che:

- $a_i : b_l > b_j$
- $b_l : a_i > a_k$



Per contraddizione, supponiamo che $\exists \{a_i; b_j\}, \{a_k, b_l\} \in M$ che formino un'instabilità. Sappiamo, senza perdita di generalità, che:

1. $a_i : b_l > b_j$ (a_i preferisce b_l a b_j)
2. $b_l : a_i > a_k$

Per come funziona l'algoritmo, l'ultima proposta di a_i sarà fatta a b_j . Ne derivano due possibili scenari:

1. a_i si è proposto a b_l prima di b_j . Allora visto che b_l ha accettato la proposta di a_k (per ipotesi contraddittoria), vuol dire che $b_l : a_k > a_i$, il che è in contraddizione con l'ipotesi.
2. a_i non si è proposto a b_l , quindi a_i ha dato le sue proposte in ordine decrescente di preferenza. Quindi $a_i : b_j > b_l$, il che è in contraddizione con l'ipotesi.

Quindi M non contiene instabilità e quindi è stabile. L'algoritmo di Gale-Shapley termina con un matching stabile. $\square$

Definizione 1.13. Diciamo che $b \in B$ è un matching valido per un certo $a \in A$ se uno esiste uno stable matching M tale che $\{a, b\} \in M$.

Definizione 1.14. Sia $best(a_i)$, per ogni $a_i \in A$, il matching valido per a_i . Per cui a_i ha la massima preferenza tra tutti i matching validi per a_i .

Teorema 1.15. *L'algoritmo di Gale-Shapley restituisce il matching:*

$$M^* = \{\{a_i, best(a_i)\} \mid \forall a_i \in A\}$$

Definizione 1.16. Diciamo che $a \in A$ è un match valido per un certo $b \in B$ se esiste uno stable matching M tale che $\{a, b\} \in M$.

Definizione 1.17. Sia $worst(b_i)$, per ogni $b_i \in B$, il match valido per b_i . Per cui b_i ha la minima preferenza tra tutti i matching validi per b_i .

Teorema 1.18. *L'algoritmo di Gale-Shapley restituisce il matching:*

$$M^* = \{\{worst(b_i), b_i\} \mid \forall b_i \in B\}$$

1.2 Notazioni asintotiche

Diciamo che $T(n)$ è il tempo di esecuzione di un certo algoritmo sull'input di dimensione n su cui è più "lento". Ad esempio nel problema 1.2 il running time non è fisso, dato che dipende dall'istanza: se io cerco l'elemento che si trova nella prima posizione del vettore l'algoritmo termina subito, mentre se cerco un elemento che non è presente allora l'algoritmo termina dopo aver esaminato tutti gli elementi del vettore. L'algoritmo può quindi richiedere fino a n iterazioni.

Definizione 1.19. Se $T(n)$ e $f(n)$ sono funzioni non-negative e crescenti, diciamo che:

$$\begin{aligned} T(n) &\text{ è } \mathcal{O}(f(n)) \\ T(n) &= \mathcal{O}(f(n)) \\ T(n) &\leq \mathcal{O}(f(n)) \\ T(n) &\in \mathcal{O}(f(n)) \end{aligned}$$

Se $\exists c, n_0 > 0$ tali che $\forall n > n_0 : T(n) \leq c \cdot f(n)$

Esempio 1.20. L'algoritmo ha running time $\mathcal{O}(n)$, tuttavia l'input, sebbene è n , è logaritmico

```

1: function PRIME( $n$ )
2:   for  $i \leftarrow 2$  to  $\lfloor \frac{n}{2} \rfloor$  do
3:     if  $n \bmod i = 0$  then
4:       return false
5:     end if
6:   end for
7:   return true
8: end function

```

rispetto al numero di bit dell'input. Infatti, se n è un numero di k bit, allora $n \leq 2^k$. Quindi il running time dell'algoritmo è $\mathcal{O}(2^k)$.

Definizione 1.21. Se $T(n)$ e $f(n)$ sono funzioni non-negative e crescenti, diciamo che:

$$T(n) \text{ è } \Omega(f(n))$$

$$T(n) = \Omega(f(n))$$

$$T(n) \geq \Omega(f(n))$$

$$T(n) \in \Omega(f(n))$$

Se $\exists c, n_0 > 0$ tali che $\forall n > n_0 : T(n) > c \cdot f(n)$

Definizione 1.22. Se $T(n)$ e $f(n)$ sono funzioni non-negative e crescenti, diciamo che:

$$T(n) \text{ è } \Theta(f(n))$$

$$T(n) = \Theta(f(n))$$

$$T(n) \in \Theta(f(n))$$

Se $T(n) \in \mathcal{O}(f(n))$ e $T(n) \in \Omega(f(n))$.

Ora studiamo un modo intelligente di costruire l'algoritmo di Gale Shapley 3 ricordando i passi:

1. Identificare un elemento libero a_i
2. Identificare il b_j che è più in alto nella lista di preferenze di a_i , e a cui a_i non ha ancora proposto
3. Verificare se b_j sia libero e, in caso negativo, trovare il suo partner attuale a_k
4. Verificare se b_j preferisce a_i ad a_k

Una struttura dati intelligente cambia drasticamente il running time di un algoritmo. Ad esempio per il punto 4 si può pensare ad array di questo genere

$$b_j : \begin{array}{|c|c|c|c|} \hline 1 & 2 & 3 & 4 \\ \hline a_3 & a_1 & a_4 & \dots \\ \hline \end{array} \quad b_j : \begin{array}{|c|c|c|c|} \hline a_1 & a_2 & a_3 & a_4 \\ \hline 2 & 4 & 1 & 3 \\ \hline \end{array}$$

Per implementare le 4 operazioni in tempo costante $\mathcal{O}(1)$ faremo uso delle strutture dati:

- A. **apref**: un array $n \times n$ che contiene in **apref** $[i][j]$ l'indice dell' l -esimo partner più preferito di a_i :

$$a_3 : b_2 > b_1 > b_4 > b_3$$

$$\text{apref}[3][1] = 2$$

$$\text{apref}[3][2] = 1$$

$$\text{apref}[3][3] = 4$$

$$\text{apref}[3][4] = 3$$

- B. **next**: un array di dimensione n , che contiene in **next** $[i]$ la posizione (il rango) nell'ordine di preferenza di a_i , del prossimo b_j cui a_i si proporrà. Quando a_i si propone, si proporrà a b_j dove $j = \text{apref}[i][\text{next}[i]]$. Dopo una proposta incrementiamo **next** $[i]$

Per cui A e B ci permettono di implementare il punto 2 in tempo $\mathcal{O}(1)$

- C. **current**: un array di dimensione n , tale che **current** $[i]$ contiene il j tale che b_i e a_j sono attualmente accoppiati. Se b_i fosse libera, avremmo **current** $[i] = \text{None}$

C ci permette di implementare il punto 3 in tempo $\mathcal{O}(1)$

- D. **ranking**: un array $n \times n$ tale che **ranking** $[j][i]$ è la posizione (o il rango) di a_i nella lista di preferenze di b_j .

$$b_3 : a_2 > a_1 > a_3 > a_4$$

```

    ranking[3][2] = 1
    ranking[3][1] = 2
    ranking[3][3] = 3
    ranking[3][4] = 4

```

D ci permette di implementare il punto 4 in tempo $\mathcal{O}(1)$. Per il punto 1 necessitiamo di una struttura dati che ci permette di estrarre e inserire gli oggetti in tempo costante. Utilizzeremo una lista concatenata, di cui qui sotto si riporta il codice python:

```

1 class Node:
2     def __init__(self, i):
3         self.x = i
4         self.next = None

```



Figura 1: Rappresentazione di una lista concatenata

Questa lista ci permette di estrapolare in tempo costante il primo elemento. Vediamo quindi come inizializzare la lista dei nodi:

```

1 def Init():
2     list = Node(1)
3     for i in range(2, n+1):
4         N = Node(i)
5         N.next = list
6         list = N
7     return list

```

E quindi abbiamo costruito la lista concatenata in tempo $\mathcal{O}(n)$. Dobbiamo quindi definire un modo sia per estrarre un elemento sia per inserirlo (entrambi in tempo costante):

```

1 def Extract(list):
2     N = list
3     list = list.next
4     return N.x, list    # N.x restituisce indice

```

```

1 def Add(x, list):
2     N = Node(x)
3     N.next = list
4     list = N
5     return list

```

Possiamo quindi modificare l'inizializzazione della lista concatenata utilizzando la funzione che abbiamo appena definito:

```

1 def Init():
2     list = None
3     for i in range(1, n+1):
4         list = Add(i, list)
5     return list

```

Per cui abbiamo dimostrato il seguente lemma:

Lemma 1.23. *Ogni iterazione del ciclo di Gale-Shapley può essere implementata in tempo $\mathcal{O}(1)$. Solo l'inizializzazione richiede $\mathcal{O}(n)$, ma viene eseguito una sola volta.*

Chiaramente a seconda dell'input l'algoritmo potrebbe metterci un tempo inferiore a $\mathcal{O}(n^2)$.

Teorema 1.24. *Gale-Shapley può essere implementato in tempo $\mathcal{O}(n^2)$*

2 Algoritmi Greedy

2.1 Interval Scheduling

Si dispone di una risorsa e di n “job”. Ogni job_i ha un tempo di inizio s_i e uno di fine f_i . Si vuole trovare un sottoinsieme di job di cardinalità massima e composto da job compatibili.

Definizione 2.1. Un insieme di job è compatibile se $\forall (s_i, f_i), (s_j, f_j) \in S$ con $i \neq j$ si ha che $\min(f_i, f_j) \leq \max(s_i, s_j)$.

Ogni istante di inizio e fine è etichettato come s_i e f_i rispettivamente.

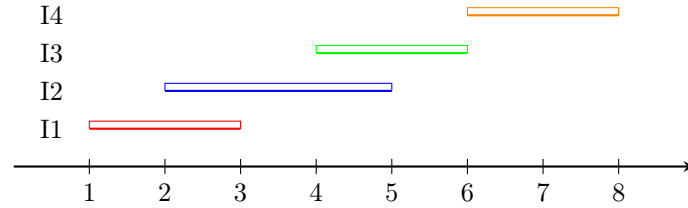


Figura 2: Rappresentazione grafica delle attività nel problema di Interval Scheduling. Le attività selezionate sono evidenziate in verde e etichettate con s_i e f_i , mentre quelle non selezionate sono in rosso.

Quindi data un'istanza I si trovi $S \subseteq I$ tale che S è compatibile e $|S|$ è massimo. L'algoritmo opera attraverso un approccio greedy, selezionando ad ogni passo l'attività che termina prima. Questo è uno scheletro di un algoritmo greedy, a cui manca la definizione della regola M . Nella

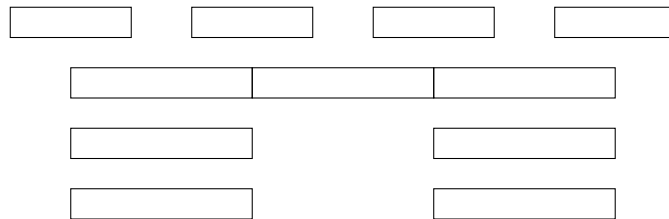
```

1: function SELECTM( $I$ )
2:    $S \leftarrow \emptyset$ 
3:   while  $I \neq \emptyset$  do
4:     Si selezioni un intervallo  $(s_j, f_j) \in I$  secondo la regola  $M$ 
5:      $N = \{(s_i, f_i) \mid (s_i, f_i) \in I \wedge (s_j, f_j), (s_i, f_i) \text{ sono incompatibili}\}$ 
6:      $I = I \setminus N$ 
7:      $S = S \cup \{(s_j, f_j)\}$ 
8:   end while
9:   return  $S$ 
10: end function

```

pratica, l'approccio greedy è molto rapido, tuttavia non è in grado di risolvere tutti i problemi. Vediamo quindi una serie di M :

- $M =$ “si prenda l'intervallo di lunghezza minima”
- $M =$ “si prenda un intervallo incompatibile con il minor numero di intervalli rimanenti”. È presentata sotto un'illustrazione che mostra come questa regola non può essere universalmente ottima seppur ragionevole.



- $M^\star =$ “si prenda un intervallo tra quelli che terminano il prima possibile”.

Teorema 2.2. *Select_M restituisce una soluzione ottima*

Lemma 2.3. *Per ogni regola M , Select_M restituisce un insieme compatibile di intervalli (una soluzione ammissibile/feasible)*

L'idea che sta dietro alla dimostrazione di questo lemma è che ogni intervallo in I è compatibile con ogni intervallo in S . Procediamo ora con la dimostrazione del teorema.

Dimostrazione. Sia $O \subseteq I$ una soluzione ottima (O è compatibile, e $|O|$ è massima). Senza perdere di generalità sia $O = \{J_1, J_2, \dots, J_m\}$ con $s(J_i)$ uguale all'inizio dell'intervallo J_i e $f(J_i)$ uguale alla fine dello stesso intervallo. Supponiamo, sempre s.p.d.g., che $f(J_1) \leq f(J_2) \leq \dots \leq f(J_m)$. Chiaramente $s(J_i) \leq f(J_i)$ e $f(J_i) \leq s(J_{i+1}) \forall i \in \{1, 2, \dots, m-1\}$. Quindi si ha:

$$s(J_1) \leq f(J_1) \leq s(J_2) \leq f(J_2) \leq \dots \leq s(J_m) \leq f(J_m)$$

Sia S la soluzione restituita da Select_{M^*} . Per il lemma precedente, S è compatibile, quindi, s.p.d.g., $S = \{I_1, I_2, \dots, I_k\}$ e:

$$s(I_1) \leq f(I_1) \leq s(I_2) \leq f(I_2) \leq \dots \leq s(I_k) \leq f(I_k)$$

Vogliamo mostrare che $k \geq m$, cioè che $|S| \geq |O|$ (e che quindi $k = m$ per cui $|S| = |O|$). Si vuole affermare che:

- $f(I_1) \leq f(J_1)$
Per la regola M^* , I_1 è un intervallo di quelli che finiscono per primi. Quindi non può esistere un intervallo J_1 che finisce prima di I_1 . Quindi se $O = \{J_1, J_2, \dots, J_m\}$ è una soluzione ottima, allora $O_1 = \{I_1, J_2, J_3, \dots, J_m\}$ è una soluzione ottima (O_1 è compatibile dato che $f(I_1) \leq f(J_1)$ e $|O_1| = |O| = m$)
- $\forall 1 \leq i \leq k \exists$ una soluzione compatibile $O_i = \{I_1, I_2, \dots, I_i, J_{i+1}, J_{i+2}, \dots, J_m\}$ tale che $f(I_i) \leq f(J_i)$
Abbiamo già dimostrato il caso $i = 1$. Per induzione, assumiamo l'asserto per $i \leq k-1$, e lo dimostriamo per $i+1$. Per induzione $O_i = \{I_1, I_2, \dots, I_i, J_{i+1}, J_{i+2}, \dots, J_m\}$ è una soluzione ottima con $f(I_i) \leq f(J_i)$.
Definiamo $O_{i+1} = \{I_1, I_2, \dots, I_i, I_{i+1}, J_{i+2}, J_{i+3}, \dots, J_m\}$ allora $|O_{i+1}| = |O_i| = m$
 Select_M ha selezionato I_{i+1} come $i+1$ esimo intervallo. Quindi I_{i+1} è un intervallo che finisce per primo tra quelli compatibili con ciascuno degli intervalli $I_1, I_2, \dots, I_i$. Quindi $f(I_{i+1}) \leq f(J_{i+1})$ (J_{i+1} è infatti compatibile con ciascuno degli intervalli $I_1, I_2, \dots, I_i$) e O_{i+1} è compatibile.

Consideriamo allora $O_k = \{I_1, I_2, \dots, I_k, J_{k+1}, \dots, J_m\}$. Allora, $O_k \supseteq S$, cioè O_k ha la soluzione di greedy come sottoinsieme. Select_{M^*} termina quando non esistono intervalli compatibili con quelli già selezionati. Quindi, nessuno di $J_{k+1}, J_{k+2}, \dots, J_m$ può esistere. Segue che $O_k = \{I_1, I_2, \dots, I_k\}$, e quindi che $k = m$. Quindi Select_{M^*} restituisce una soluzione ottima. □

2.2 Interval Partitioning

Problema 2.1. Abbiamo un insieme I di intervalli. Qual'è il minimo numero di risorse necessarie per eseguirli tutti.

Definizione 2.4. Data un'istanza I , definiamo $\text{Depth}(I)$ il minimo intero d tale che $\forall t \in \mathbb{R}$

$$|\{I_i \mid I_i \in I \wedge t \in I_i\}| \leq d$$

Definizione 2.5. Sia $\text{Opt}(I)$ il minimo numero di risorse per schedulare ogni intervallo di I . Vale che:

$$\text{Opt}(I) \geq \text{Depth}(I)$$

Dimostrazione. Deve esistere un tempo t in cui devono essere eseguiti $\text{Depth}(I)$ intervalli. Altrimenti $\text{Depth}(I)$ sarebbe inferiore al suo intervallo. Al tempo t dobbiamo quindi schedulare almeno $\text{Depth}(I)$ intervalli, e per farlo abbiamo bisogno di almeno $\text{Depth}(I)$ risorse. □

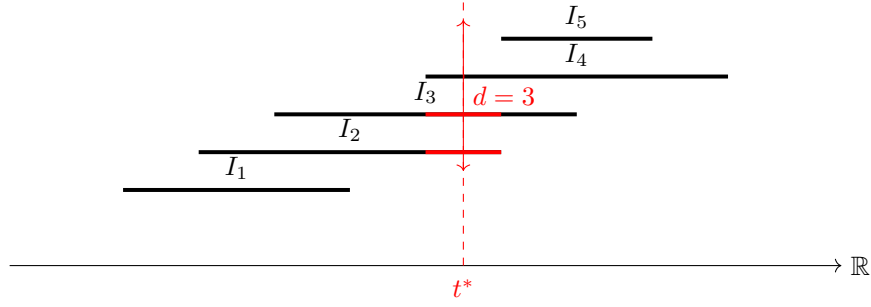


Figura 3: Illustrazione del concetto di $\text{Depth}(I)$: al punto t^* la profondità è $d = 3$.

Vediamo la struttura di un primo algoritmo per il problema di Interval Partitioning:

- Sia $d = \text{Depth}(I)$ (tempo $n \log n$)
- Si ordinino gli intervalli per tempo di inizio in modo crescente
- Sia $I = \{(s_1, f_1), (s_2, f_2), \dots, (s_n, f_n)\}$ con $s_1 \leq s_2 \leq \dots \leq s_n$

```

1: function PARTITION1( $I$ )
2:    $d = \text{Depth}(I)$ 
3:   Si ordinino gli intervalli per tempo di inizio in modo crescente
4:   for  $j = 1$  to  $n$  do
5:      $L \leftarrow \{1, 2, \dots, d\}$ 
6:     for  $i = 1$  to  $j - 1$  do
7:       if  $(s_i, f_i)$  e  $(s_j, f_j)$  sono incompatibili then
8:          $L \leftarrow L - \{l(i)\}$   $\triangleright l(i)$  è la risorsa (etichettata) assegnata ad  $i$ 
9:       end if
10:    end for
11:    if  $|L| \geq 1$  then
12:      Sia  $a$  un elemento arbitrario di  $L$  per cui  $l(j) = a$ 
13:    else
14:      l'algoritmo fallisce
15:    end if
16:  end for
17:  return  $l(1), l(2), \dots, l(n)$ 
18: end function

```

Lemma 2.6. *Se l'algoritmo non fallisce allora restituisce una soluzione valida (feasible). Il ciclo interno fa sì che non sia mai assegnata la stessa etichetta a due intervalli incompatibili.*

Lemma 2.7. *Se l'algoritmo non fallisce, restituisce una soluzione ottima*

Dimostrazione. Per il primo lemma, ogni soluzione valida deve usare almeno $\text{Depth}(I)$ risorse. Se l'algoritmo non fallisce, restituisce una soluzione valida (2.6) fatta di $\text{Depth}(I)$ risorse. $\square$

Lemma 2.8. *L'algoritmo non fallisce mai.*

Dimostrazione. Si consideri una generica interazione $\hat{j}$ del ciclo esterno. Sia $S_{\hat{j}}$ l'insieme degli intervalli che:

1. Sono stati considerati dall'algoritmo prima di $(s_{\hat{j}}, f_{\hat{j}})$
2. Finiscono dopo $s_{\hat{j}}$

Quindi, $S_{\hat{j}} = \{s_i, f_i\}$ con $i \leq \hat{j} - 1$ e $f_i > s_{\hat{j}}$

$\square$

L'algoritmo, nell'iterazione $\hat{j}$, rimuoverà da $L = \{1, 2, \dots, d\}$ tutte e sole le etichette degli intervalli di $S_{\hat{j}}$. Quindi, dopo la fine dell'esecuzione del **for** interno, avremo che $|L| \geq d - |S_{\hat{j}}|$. L'obiettivo è dimostrare che, dopo il **for** interno, $|L| \geq 1$. Per farlo dimostreremo che $|S_{\hat{j}}| \leq d - 1$.

Dimostrazione. Ogni intervallo in $S_{\hat{j}}$ è attivo all'istante $s_{\hat{j}}$ (ogni tale intervallo inizia non oltre $s_{\hat{j}}$, e finisce dopo). Per contraddizione, supponiamo che $|S_{\hat{j}}| \geq d$. Osserviamo che $(s_{\hat{j}}, f_{\hat{j}}) \notin S_{\hat{j}}$. Quindi $|S_{\hat{j}} \cup \{s_{\hat{j}}, f_{\hat{j}}\}| \geq d + 1$. Visto che ogni intervallo di $S_{\hat{j}}$ è attivo al tempo $s_{\hat{j}}$, e che $d = \text{Depth}(I)$, è impossibile che $|S_{\hat{j}} \cup \{s_{\hat{j}}, f_{\hat{j}}\}| \geq d + 1 = \text{Depth}(I) + 1$. $\square$

Segue che $|L| \geq 1$, alla fine dell'esecuzione del ciclo interno ad ogni iterazione del ciclo esterno. Quindi l'algoritmo non fallisce mai.

Teorema 2.9. *L'algoritmo Partition_1 restituisce un assegnamento ottimo.*

L'algoritmo Partition_1 ha running time $\mathcal{O}(n^2)$ data la presenza di due cicli annidati. Si potrebbe efficientare ulteriormente l'algoritmo utilizzando un heap binario, come segue:

Algorithm 4 $\text{Partition}_{\text{Optimized}}$ usando Heap Binaria

```

1: function PARTITION_OPTIMIZED( $I$ )
2:   Input: Insieme di intervalli  $I = \{(s_1, f_1), (s_2, f_2), \dots, (s_n, f_n)\}$ 
3:   Output: Assegnazione delle risorse  $l(1), l(2), \dots, l(n)$ 
4:   Ordina  $I$  in ordine crescente di  $s_j$ 
5:    $d \leftarrow \text{Depth}(I)$  ▷ Calcola la profondità massima
6:   Initialize  $\text{occupied\_heap} \leftarrow$  heap vuota ▷ Min-heap basato su  $f_i$ 
7:   Initialize  $\text{available\_heap} \leftarrow \{1, 2, \dots, d\}$  ▷ Min-heap delle risorse disponibili
8:    $l \leftarrow$  array vuoto di dimensione  $n$ 
9:   for  $j = 1$  to  $n$  do
10:     $(s_j, f_j) \leftarrow I[j]$ 
11:    while not  $\text{occupied\_heap.is Empty}()$  and  $\text{occupied\_heap.min().end} \leq s_j$  do
12:       $(\text{end}, \text{res}) \leftarrow \text{occupied\_heap.extractMin}()$ 
13:       $\text{available\_heap.insert}(\text{res})$ 
14:    end while
15:    if not  $\text{available\_heap.is Empty}()$  then
16:       $a \leftarrow \text{available\_heap.extractMin}()$ 
17:       $l[j] \leftarrow a$ 
18:    else
19:      fail ▷ Non ci sono risorse disponibili
20:    end if
21:     $\text{occupied\_heap.insert}((f_j, l[j]))$ 
22:  end for
23:  return  $l[1], l[2], \dots, l[n]$ 
24: end function

```

2.3 Compressione dei Dati

Affronteremo il problema della compressione di un testo: una stringa di n caratteri maiuscoli ('A', 'B', ..., 'Y', 'zZ', ' ', ';', ','). Con la codifica ASCII vengono usati 8 bit per carattere. La nostra stringa di n caratteri richiede quindi $8n$ bit per essere rappresentata con ASCII.

$A \rightarrow 0 \Rightarrow 00000000$
 $B \rightarrow 1 \Rightarrow 00000001$
 $\vdots$
 $Z \rightarrow 25 \Rightarrow 11000$
 $\rightarrow 26 \Rightarrow 11001$
 $;\rightarrow 27 \Rightarrow 11010$

Con una codifica (fissa) a 5 bit per carattere, riusciamo a rappresentare la stringa con $5n$ bit. Questo è un esempio di compressione lossless. Si può dimostrare che se la nostra stringa è generata in maniera casuale, allora la codifica a 5 bit è ottima. In italiano, le frequenze empiriche dei caratteri sono:

$$\begin{aligned} E &\sim 11.79\% \\ A &\sim 11.74\% \\ I &\sim 11.28\% \\ O &\sim 9.83\% \\ N &\sim 6.88\% \\ &\vdots \\ Z &\sim 0.49\% \end{aligned}$$

Data la non-uniformità delle frequenze, non è molto ragionevole usare lo stesso numero di bit per ogni lettera.

Definizione 2.10 (Prefix-Coding). Un Prefix-Code per un insieme S di lettere è una funzione γ da S alle stringhe di bit, tale che $\forall x, y \in S \ x \neq y$ implica che $\gamma(x)$ non sia prefisso di $\gamma(y)$.

Esempio 2.11. Se usiamo la seguente codifica:

$$\begin{aligned} A &\rightarrow 0 \\ B &\rightarrow 00 \end{aligned}$$

Non abbiamo un Prefix-Code, in quanto $\gamma(A)$ è prefisso di $\gamma(B)$. E quindi la codifica non è univoca: la stringa 0000 può essere interpretata come AAAA, BB, ABA, AAB, BAA

Esempio 2.12. Usiamo adesso la seguente codifica:

$$\begin{aligned} A &\rightarrow 0 \\ B &\rightarrow 10 \\ C &\rightarrow 11 \end{aligned}$$

Questa è una codifica univoca, e quindi è un Prefix-Code.

Per rappresentare una sequenza di lettere $X = x_1, x_2, \dots, x_n$, possiamo usare la stringa di bit $\Gamma = \gamma(x_1)\gamma(x_2)\dots\gamma(x_n)$. Questa rappresentazione è univoca, e quindi possiamo decodificare la stringa di bit per ottenere la sequenza originale.

$$ABC \rightarrow 0\ 10\ 11 \quad CBC \rightarrow 11\ 10\ 11$$

Come possiamo arrivare ad x partendo da Γ

- Scansioniamo Γ da sinistra a destra
- Appena arriviamo ad un prefisso di Γ che è uguale alla codifica $\gamma(x)$ di una lettera $x \in S$:
 - Diamo in output x
 - Cancelliamo il prefisso $\gamma(x)$ da Γ

Lemma 2.13. Questa decodifica è corretta se γ è un Prefix-Code.

Esempio 2.14. Sia $S = \{A, B, C, D, E\}$

$$\begin{aligned} \gamma_1(A) &= 11 \\ \gamma_1(B) &= 01 \\ \gamma_1(C) &= 001 \\ \gamma_1(D) &= 10 \\ \gamma_1(E) &= 000 \end{aligned}$$

Prefix Code Ottimo Supponiamo che le frequenze siano:

$$f_A = 0.32 \quad f_b = 0.25 \quad f_C = 0.2 \quad f_D = 0.18 \quad f_E = 0.05$$

Il costo medio (in bit) di una lettera sarà:

$$0.32 \cdot 2 + 0.25 \cdot 2 + 0.2 \cdot 3 + 0.18 \cdot 2 + 0.05 \cdot 3 = 2.25$$

Osserviamo che γ_1 non è ottimo per le frequenze date. Un Prefix-Code ottimo per le frequenze date è:

$$\begin{aligned}\gamma_2(A) &= 11 \\ \gamma_2(B) &= 01 \\ \gamma_2(C) &= 10 \\ \gamma_2(D) &= 001 \\ \gamma_2(E) &= 000\end{aligned}$$

Il costo medio di una lettera con γ_2 è:

$$0.32 \cdot 2 + 0.25 \cdot 2 + 0.2 \cdot 2 + 0.18 \cdot 3 + 0.05 \cdot 3 = 2.23 < 2.25$$

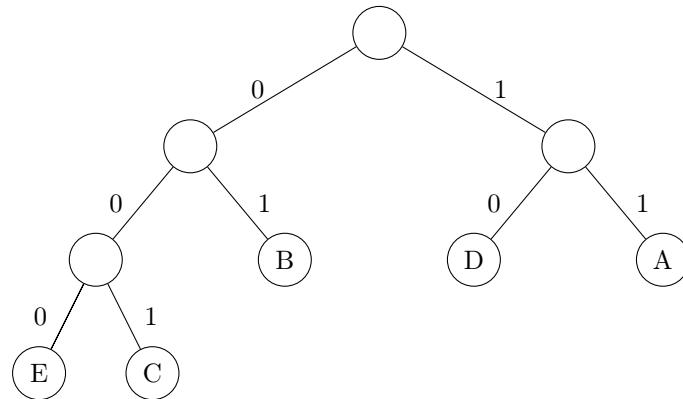
Problema 2.2. Dato un alfabeto S , con le frequenze delle sue lettere, si trovi un prefix-code γ che minimizza la $ABL(\gamma)$ (Average Bit Length).

$$ABL(\gamma) = \sum_{x \in S} (f_x \cdot |\gamma(x)|)$$

Lo spazio delle soluzioni è piuttosto complicato (quali sono le proprietà di un Prefix-Code? Come possiamo usarle per sviluppare un algoritmo efficiente?) e grande.

2.4 Prefix-Code e alberi binari

Sia T un albero binario in cui, ad ogni foglia, associamo una lettera distinta di S . Quando si attraversa l'albero da radice a foglia, si assegna un bit a ciascun arco, e quando si va da un nodo a un figlio sinistro si assegna 0, e 1 al figlio destro. Presa una foglia arbitraria, con etichetta $x \in S$, l'encoding di x è la sequenza di bit che si incontra nel cammino dalla radice alla foglia x .



Da cui ottengo la codifica:

$$\begin{aligned}A &\rightarrow 11 \\ B &\rightarrow 01 \\ C &\rightarrow 001 \\ D &\rightarrow 10 \\ E &\rightarrow 000\end{aligned}$$

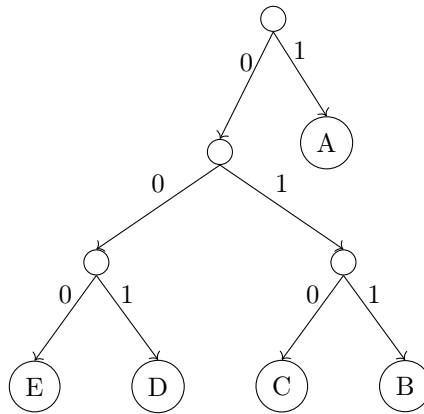
Lemma 2.15. *L'encoding di S costruito a partire dall'albero binario T è un Prefix-Code.*

Dimostrazione. Se l'encoding di $x \in S$ è un prefisso dell'encoding di $y \in S$, $y \neq x$, il cammino dalla radice a x è un prefisso del cammino dalla radice a y . Ma x è una foglia e $y \neq x$. Assurdo $\square$

Possiamo anche costruire alberi binari a partire da un Prefix-Code:

$$\begin{aligned}\gamma_0(A) &= 1 \\ \gamma_0(B) &= 011 \\ \gamma_0(C) &= 010 \\ \gamma_0(D) &= 001 \\ \gamma_0(E) &= 000\end{aligned}$$

Per cui riusciamo a rappresentare l'albero binario:



Teorema 2.16. *C'è una biezione tra gli alberi binari con etichette distinte sulle foglie ed i Prefix-Code.*

Dato un albero binario T con etichette sulle foglie, scriviamo $\text{Depth}_T(x)$ per indicare la profondità in T della foglia etichettata con x .

$$ABL(T) = \sum_{x \in S} f_x \cdot \text{Depth}_T(x)$$

Si trovi un albero binario T che minimizza $ABL(T)$. Si tratta di un albero binario in quanto ci possono essere degli alberi non binari che danno un Prefix-Code funzionante ma non ottimo.

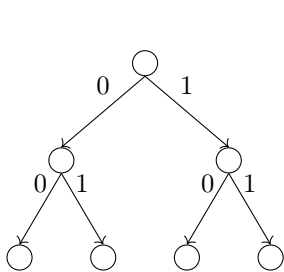


Figura 4: Albero binario ottimale

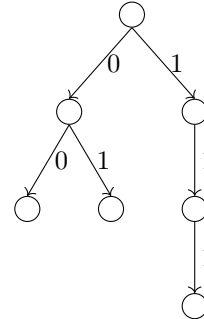
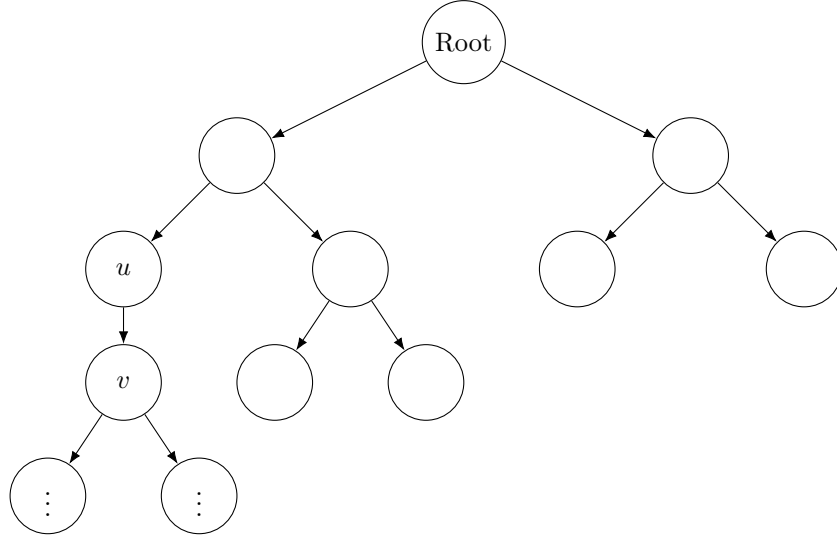


Figura 5: Albero binario non ottimale

Per trovare un albero ottimo iniziamo elencando alcune proprietà:

Lemma 2.17. *Se T è ottimo, allora è pieno. Cioè, ogni nodo interno (non una foglia) ha esattamente due figli.*

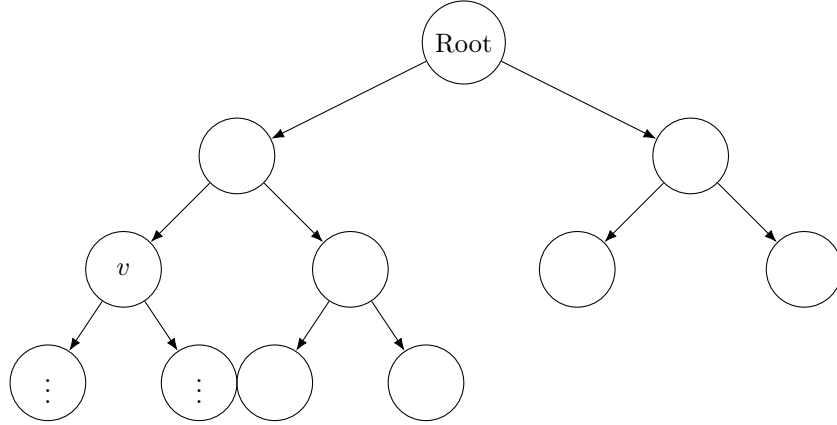
Dimostrazione. Per contraddizione, sia T ottimo e supponiamo che contenga un nodo interno u con esattamente un figlio v . Di sotto l'albero T :



Costruiamo quindi un nuovo albero T' come segue:

- Se u non è la radice, elimina u e mette v al suo posto
- Se u è la radice, elimina u e mette v come radice

Di seguito l'albero T' ottenuto:



Abbiamo quindi diminuito la profondità di alcune foglie e lasciata invariata la profondità delle altre, che possono esistere o meno. Necessariamente, visto che $f_x > 0 \forall x \in S$, si ha che $ABL(T') < ABL(T)$. Segue quindi che T non è ottimo. Assurdo. $\square$

Quindi possiamo limitare la nostra ricerca agli alberi binari pieni.

Lemma 2.18. *Se T^* è ottimo e u, v sono due foglie di T^* tali che:*

1. $\text{Depth}_{T^*}(u) < \text{Depth}_{T^*}(v)$
2. L'etichetta di u è $y \in S$ e l'etichetta di v è $x \in S$

Allora $f_y \geq f_x$

Dimostrazione. Ricordiamo che $ABL(T^*) = \sum_{z \in S} (f_z \cdot \text{Depth}_{T^*}(z))$. (uso z per indicare la foglia dell'albero a cui z è stata assegnata). Quindi

$$ABL(T^*) = f_x \cdot \text{Depth}_{T^*}(x) + f_y \cdot \text{Depth}_{T^*}(y) + \sum_{z \in S \setminus \{x, y\}} (f_z \cdot \text{Depth}_{T^*}(z))$$

(questo grazie alla linearità della sommatoria). Per assurdo, supponiamo che $f_y < f_x$. Sia $\bar{T}^*$ l'albero ottenuto da T^* scambiando le etichette x e y . $\bar{T}^*$ è ancora un albero binario. Allora:

$$\begin{aligned} ABL(\bar{T}^*) &= f_x \cdot \text{Depth}_{\bar{T}^*}(x) + f_y \cdot \text{Depth}_{\bar{T}^*}(y) + \sum_{z \in S \setminus \{x, y\}} (f_z \cdot \text{Depth}_{\bar{T}^*}(z)) \\ &= f_x \cdot \text{Depth}_{T^*}(y) + f_y \cdot \text{Depth}_{T^*}(x) + \sum_{z \in S \setminus \{x, y\}} (f_z \cdot \text{Depth}_{T^*}(z)) \end{aligned}$$

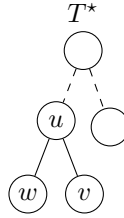
Ma allora:

$$\begin{aligned} ABL(T^*) - ABL(\bar{T}^*) &= f_x \cdot \text{Depth}_{T^*}(x) + f_y \cdot \text{Depth}_{T^*}(y) - f_x \cdot \text{Depth}_{T^*}(y) - f_y \cdot \text{Depth}_{T^*}(x) \\ &= (f_x - f_y) \cdot \text{Depth}_{T^*}(x) + (f_y - f_x) \cdot \text{Depth}_{T^*}(y) \\ &= \underbrace{(f_x - f_y)}_{>0 \text{ (per assurdo)}} \cdot \underbrace{(\text{Depth}_{T^*}(x) - \text{Depth}_{T^*}(y))}_{>0} > 0 \end{aligned}$$

Segue quindi che $ABL(\bar{T}^*) < ABL(T^*)$, per cui T^* non è ottimo. Assurdo. Quindi $f_y \geq f_x$ $\square$

Quindi, fissata la struttura dell'albero, ci conviene assegnare lettere di frequenza più alta a foglie meno profonde (l'assegnamento è quindi banale una volta fissata la struttura dell'albero).

Ora, sia v una foglia di T^* di massima profondità, sia u il padre di v e sia w l'altro figlio di u .



Lemma 2.19. w è una foglia di T^*

Dimostrazione. Se non lo fosse, l'albero radicato in w conterrebbe una foglia più profonda di v , contraddicendo la scelta di v come foglia di massima profondità. $\square$

Segue che v e w sono due foglie di massima profondità.

Teorema 2.20. *Esiste un prefix-code ottimo, con albero associato T^* , che assegna due lettere di minima frequenza a due foglie sorelle in T^* . Quindi abbiamo due lettere di frequenza minima, y e z , figlie dello stesso padre. Un algoritmo può quindi rimuovere l'alberello di altezza 1 che ha per foglie y e z e per radice il padre dei due (rappresentato giù), da T^* e sostituirgli una nuova foglia/lettera ω (dove ω è una nuova meta-lettera). Poi possiamo iterare in modo greedy.*

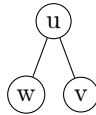


Figura 6: albero di altezza 1

Vediamo quindi l'algoritmo di Huffman, che costruisce un prefix-code ottimo:

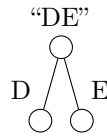
Algorithm 5 Huffman

```
1: function HUFFMAN( $S, f$ )
2:   if  $|S| = 2$  then
3:     Sia  $S = \{y^*, z^*\}$ 
4:     Si codifichi una delle due lettere con 1 e l'altra con 0
5:     Sia  $T =$  albero in figura 6
6:   else
7:     Siano  $y^*$  e  $z^*$  due lettere di frequenza più bassa in  $S$ 
8:     Sia  $S' = S - \{y^*, z^*\} \cup \{\omega_{y^*, z^*}\}$ 
9:     Sia  $f'$  tale che  $f'_a = f_a \ \forall a \in S \setminus \{y^*, z^*\}$  e inoltre  $f'_{\omega_{y^*, z^*}} = f_{y^*} + f_{z^*}$ 
10:    Ricorsivamente, costruisco un prefix-code ottimo per  $(S', f')$  con albero associato  $T'$ 
11:    Creo un albero  $T$  sostituendo in  $T'$  la foglia  $\omega_{y^*, z^*}$  con albero in figura 6
12:  end if
13:  return  $T$ 
14: end function
```

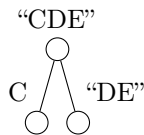
L'algoritmo di Huffman restituisce un huffman-code

Esempio 2.21.

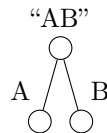
$$f_A = 0.32 \quad f_B = 0.25 \quad f_C = 0.2 \quad f_D = 0.18 \quad f_E = 0.05$$



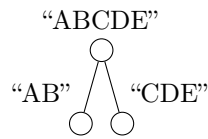
$$f_A = 0.32 \quad f_B = 0.25 \quad f_C = 0.2 \quad f_{DE} = 0.23$$



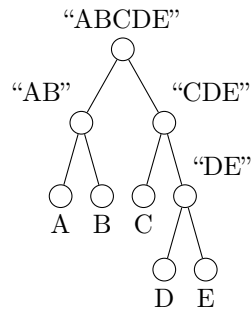
$$f_A = 0.32 \quad f_B = 0.25 \quad f_{CDE} = 0.43$$



$$f_{AB} = 0.57 \quad f_{CDE} = 0.43$$



Combinando le soluzioni dei sottoalberi si ottiene il seguente albero:



Quindi si avranno i seguenti encoding:

$$\begin{aligned} A &\rightarrow 00 \\ B &\rightarrow 01 \\ C &\rightarrow 10 \\ D &\rightarrow 110 \\ E &\rightarrow 111 \end{aligned}$$

E si può quindi ricostruire l'albero di Huffman di partenza. Se $Y^*, Z^* \in S$ sono due lettere di minima frequenza, l'algoritmo:

1. Le sostituisce con una nuova lettera ω dove $f_\omega = f_{y^*} + f_{z^*}$
2. Crea un nuovo albero $S' = S - \{y^*, z^*\} \cup \{\omega\}$
3. Crea un albero T' per S' con le sue frequenze
4. Sostituisce ω in T' con un alberello di altezza 1 con foglie y^* e z^* , ottenendo l'albero T per S

Lemma 2.22.

$$ABL(T') = ABL(T) - f_\omega$$

Dimostrazione. La profondità di ogni $x \in S \setminus \{y^*, z^*\}$ è identica in T e T' . Viceversa, la profondità

di $x \in \{y^*, z^*\}$ in T è uguale alla profondità di ω in $T' + 1$.

$$\begin{aligned}
ABL(T) &= \sum_{x \in S} f_x \cdot \text{Depth}_T(x) \\
&= f_{y^*} \cdot \text{Depth}_T(y^*) + f_{z^*} \cdot \text{Depth}_T(z^*) + \sum_{x \in S \setminus \{y^*, z^*\}} f_x \cdot \text{Depth}_T(x) \\
&= f_{y^*} \cdot (1 + \text{Depth}_{T'}(\omega)) + f_{z^*} \cdot (1 + \text{Depth}_{T'}(\omega)) + \sum_{x \in S \setminus \{y^*, z^*\}} f_x \cdot \text{Depth}_{T'}(x) \\
&= f_{y^*} + f_{y^*} \cdot \text{Depth}_{T'}(\omega) + f_{z^*} + f_{z^*} \cdot \text{Depth}_{T'}(\omega) + \sum_{x \in S \setminus \{y^*, z^*\}} f_x \cdot \text{Depth}_{T'}(x) \\
&= (f_{y^*} + f_{z^*}) + (f_{y^*} + f_{z^*}) \cdot \text{Depth}_{T'}(\omega) + \sum_{x \in S \setminus \{y^*, z^*\}} f_x \cdot \text{Depth}_{T'}(x) \\
&= f_\omega + f_\omega \cdot \text{Depth}_{T'}(\omega) + \sum_{x \in S \setminus \{y^*, z^*\}} f_x \cdot \text{Depth}_{T'}(x) \\
&= f_\omega + \sum_{x \in S \setminus \{y^*, z^*\} \cup \{\omega\}} f_x \cdot \text{Depth}_{T'}(x) \\
&= f_\omega + \sum_{x \in S'} f_x \cdot \text{Depth}_{T'}(x) \\
&= f_\omega + ABL(T')
\end{aligned}$$

□

Teorema 2.23. *L'algoritmo di Huffman restituisce un prefix-code di minima ABL (quindi un prefix-code ottimo).*

Dimostrazione. Dimostriamo per induzione su $|S|$. Se $|S| \leq 2$, l'albero restituito è chiaramente ottimo (usa 1 bit per ogni lettera). Altrimenti, $|S| \geq 3$, e sia T l'albero restituito dall'algoritmo di Huffman. Per assurdo, immaginiamo che esista un albero Z tale che $ABL(Z) < ABL(T)$. Possiamo assumere che y^* e z^* (due lettere di frequenza minima) siano sorelle in Z . Se cancelliamo y^* e z^* da Z ed etichettiamo il loro padre con ω , otteniamo un albero Z' che rappresenta un prefix-code per l'alfabeto $S \setminus \{y^*, z^*\} \cup \{\omega\}$. (il modo in cui otteniamo Z' da Z è lo stesso con cui l'algoritmo ha ottenuto T a partire dal T' restituito dalla chiamata ricorsiva).

Per il lemma 2.22, $ABL(T') = ABL(T) - f_\omega$; analogamente $ABL(Z') = ABL(Z) - f_\omega$. Per contraddizione abbiamo assunto che $ABL(Z) < ABL(T)$ ma, allora:

$$ABL(Z') = ABL(Z) - f_\omega < ABL(T) - f_\omega = ABL(T')$$

Visto che T' è l'albero restituito dal nostro algoritmo su un alfabeto di cardinalità $|S'| = |S| - 1$, sappiamo per induzione che T' è ottimo. Quindi non può esistere Z' tale che $ABL(Z') < ABL(T')$. Contraddizione □

Complessità computazionale

L'algoritmo di Huffman ha una complessità $O(n^2)$, tuttavia può essere efficientato implementando un Heap. Ricordiamo i metodi di una struttura heap/priority queue:

- **add(k, v)** aggiunge alla priority queue l'elemento (chiave **k**, valore **v**) $O(\log n)$
- **remove()** rimuove e restituisce l'elemento di chiave minima $O(\log n)$
- **initialize()** inizializza la priority queue con gli elementi $x = [(k_1, v_1), (k_2, v_2), \dots, (k_i, v_i)]$ $O(i)$

E possiamo usare le frequenze come chiavi. In questo modo Huffman trova un prefix-code ottimo in tempo $n \log n$

3 Programmazione Dinamica

3.1 Weighted Interval Scheduling

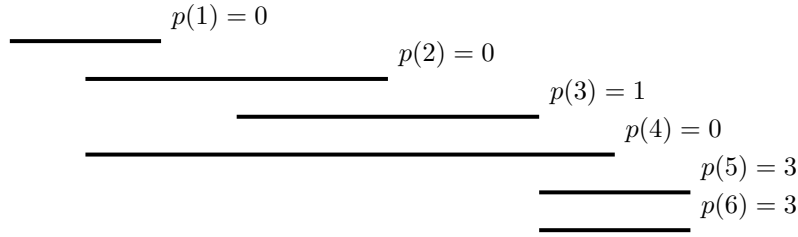
Problema 3.1. Dato un insieme di intervalli $I = \{I_1, I_2, \dots, I_m\} = \{(s_1, f_1), (s_2, f_2), \dots, (s_m, f_m)\}$ ed una pesatura degli intervalli $w(I_j) = w_j \geq 0 \forall j \in \{1, \dots, m\}$. L'obiettivo è trovare un sottoinsieme $S \subseteq I$ tale che S non abbia sovrapposizioni di intervalli e che $\sum_{j \in S} w_j$ sia massimizzata.

La programmazione dinamica consente di mantenere attive più istanze di soluzioni parziali, e di scegliere quella migliore alla fine (greedy lavora localmente, una volta che ha aggiunto un elemento alla sua soluzione, questo non sarà più tolto).

Immaginiamo che gli intervalli siano ordinati per tempo di fine: $f_1 \leq f_2 \leq \dots \leq f_m$

Definizione 3.1. L'intervallo i viene prima di j se e solo se $f_i < f_j$

Definizione 3.2. Sia $p(j)$, per un intervallo j , il massimo indice $i < j$ tale che gli intervalli i e j sono disgiunti. Se un tale i non esiste, allora $p(j) = 0$



Lemma 3.3. L'intervallo i è compatibile con ogni intervallo $1, 2, \dots, p(i)$. Inoltre i non è compatibile con $p(i) + 1$

Il problema- i è il sottoproblema $\{I_1, \dots, I_i\}$ dell'istanza originale $\{I_1, \dots, I_m\}$ composto dai primi i intervalli (ordinati per tempo di fine). Sia O_i una soluzione ottima per il problema i . Sia OPT_i il valore di O_i :

Osservazione 3.4. 1.

$$\forall j \geq 1 \quad w_j + OPT_{p(j)} \geq OPT_{j-1} \iff \exists O_j \text{ soluzione ottima per } j \text{ tale che } I_j \in O_j$$

2.

$$OPT_{j-1} \geq w_j + OPT_{p(j)} \iff \exists O_j \text{ soluzione ottima tale che } I_j \notin O_j$$

3.

$$OPT_j = \max(w_j + OPT_{p(j)}, OPT_{j-1})$$

Infatti la soluzione O_{j-1} per il sottoproblema $j-1$ è anche una soluzione per il sottoproblema j (non per forza ottima). Quindi $OPT_j \geq OPT_{j-1}$. Inoltre, la soluzione $O_{p(j)}$ può essere trasformata nella soluzione $O_{p(j)} \cup \{I_j\}$ per il sottoproblema j . Quindi $OPT_j \geq w_j + OPT_{p(j)}$. Segue che $OPT_j \geq \max(w_j + OPT_{p(j)}, OPT_{j-1})$.

Proviamo ora che $OPT_j = \max(w_j + OPT_{p(j)}, OPT_{j-1})$. Sia O_j una soluzione **ottima** per il sottoproblema j :

- Se $I_j \notin O_j$, allora O_j è anche una soluzione ottima per il sottoproblema $j-1$. Quindi, se $I_j \notin O_j$, allora $OPT_j = OPT_{j-1}$
- Se $I_j \in O_j$, allora $O_j \setminus \{I_j\}$ è una soluzione ottima per il sottoproblema $p(j)$. Quindi, se $I_j \in O_j$, allora $OPT_j = w_j + OPT_{p(j)}$

è possibile anche optare per una soluzione iterativa.

Lemma 3.5. $M\text{-compute-Opt}(j)$ restituisce OPT_j

Lemma 3.6. $M\text{-compute-Opt}(j)$ ha complessità $O(j)$

La programmazione dinamica è una tecnica generale per risolvere problemi algoritmici. A volte esistono dinamiche efficienti, a volte no. Per avere dinamiche efficienti devono esserci 3 prerequisiti:

```

1: function M-COMPUTE-OPT( $j$ )
2:   Global  $M$ 
3:   if  $j \in M$  then:
4:     return  $M[j]$ 
5:   else
6:     if  $j = 0$  then:
7:        $M[j] = 0$ 
8:     else
9:        $M[j] = \max(w_j + M - \text{compute} - \text{Opt}(p(j)), M - \text{compute} - \text{Opt}(j - 1))$ 
10:    end if
11:    return  $M[j]$ 
12:  end if
13: end function

```

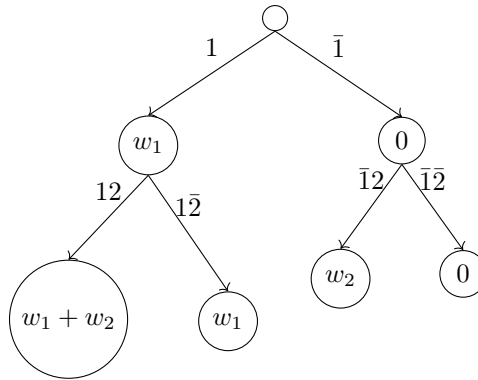
1. Devono esistere $\leq n^{O(1)}$ sottoproblemi (polinomiali).
2. La soluzione al (sotto)problema, date le soluzioni ai (sotto)sottoproblemi, deve essere computabile in tempo polinomiale.
3. I sottoproblemi devono avere un ordinamento che garantisca che la soluzione dell' i -esimo sottoproblema dipenda solo dalle soluzioni ai primi $i - 1$ sottoproblemi

La rete di sottoproblemi deve essere un grafo aciclico. In Weighted Interval Scheduling avevamo:

1. Al più 2 (sotto)sottoproblemi per (sotto)problema
2. Il tempo per computare la soluzione al sottoproblema i è $O(1)$
3. L'ordinamento è naturale (prefissi della sequenza degli intervalli ordinati per tempo di fine)

Problema 3.2. Abbiamo n jobs, l' i -esimo dei quali ha bisogno di w_i secondi per essere completato. La CPU è disponibili per W secondi ($w_1, w_2, \dots, w_n, W$ sono interi positivi). Se eseguiamo il job i veniamo pagati w_i centesimi di euro, posto che il job i termini (veniamo pagati 0.01€/s). Quale sottoinsieme di *job* mi conviene schedulare?

Potremmo optare per un approccio che decide di prendere o meno un intervallo, tuttavia la crescita è esponenziale:



Provando lo stesso approccio di Weighted Interval Scheduling, si potrebbero ordinare i job in qualche modo. Sia $OPT(i)$ il valore della soluzione ottima con a disposizione i primi i job. Se O_i è una soluzione ottima per i primi i job allora

- Se $i \notin O_i$ allora $OPT(i) = OPT(i - 1)$
- Se $i \in O_i$ non si riesce a calcolare $OPT(i)$ a partire da $OPT(1) \dots OPT(i - 1)$.

Consideriamo una diversa classe di sottoproblemi:

- Dati V ed i , qual'è il massimo valore ottenibile con V secondi di CPU, ed i primi i job?

allora

$$OPT(i, V) = \max_{\substack{S \subseteq \{1, 2, \dots, i\} \\ \sum_{j \in S} w_j \leq V}}$$

e $OPT(n, W)$ è la soluzione al mio problema originario.

Lemma 3.7. Se $i \notin O_{i,V}$, allora $OPT(i, V) = OPT(i-1, V)$.

Se $i \in O_{i,V}$, allora $OPT(i, V) = w_i + OPT(i-1, V - w_i)$ (se $w_i > V$ allora $OPT(i, V) = OPT(i-1, V)$)

Corollario 3.8. $OPT(0, V) = 0 \forall V \geq 0$. Se $i \geq 1$:

- Se $V < w_i$ allora $OPT(i, V) = OPT(i-1, V)$
- Se $V \geq w_i$ allora $OPT(i, V) = \max(OPT(i-1, V), OPT(i-1, V - w_i) + w_i)$

```

1: function SUBSET-SUM( $n, W, [w_1, \dots, w_n]$ )
2:   Si inizializzi la matrice  $M[0 \dots n][0 \dots W]$ 
3:   Sia  $M[0][V] = 0 \forall V \in \{0, 1, \dots, W\}$ 
4:   for  $i = 1$  to  $n$  do
5:     for  $V = 0$  to  $W$  do
6:       Si usi la ricorrenza del corollario per riempire  $M[i][V]$ 
7:       if  $w_i \leq V$  then
8:          $M[i][V] = \max(M[i-1][V], w_i + M[i-1][V - w_i])$ 
9:       else
10:         $M[i][V] = M[i-1][V]$ 
11:      end if
12:    end for
13:  end for
14:  return  $M[n][W]$ 
15: end function

```

per cui si trova il valore della soluzione ottima.

Lemma 3.9. L'algoritmo richiede tempo $O(nW)$

4 Machine Learning

Gli algoritmi di machine learning sono algoritmi di predizione (casi speciali di algoritmi "online"). In questo tipo di algoritmi l'input **non** è completamente disponibile all'inizio della computazione. Vediamo alcuni esempi di predizioni:

- Azioni (mercato finanziario): l'azione X aumenterà di valore in un intervallo di tempo t ?
- Eventi meteorologici
- Machine Learning: abbiamo k modelli ognuno dei quali cerca di determinare se un'immagine in input abbia la caratteristica a . A quale modello vogliamo affidarci? (ci concentreremo su quest'ultimo)

Questo tipo di algoritmo appartiene alla classe del modello degli esperti. Vediamo quindi lo scheletro del modello

	$t = 1$	$t = 2$	$t = 3$	$t = 4$
E_1	0	1	1	
E_2	1	0	1	
A	0	0	1	
R	1	0	0	

-
- 1: **for** $t = 1$ **to** T **do**
 - 2: Viene fornito all'algoritmo il vettore delle predizioni $(y_{1t}, y_{2t}, \dots, y_{nt})$, dove $y_{it} \in \{0, 1\}$ è la predizione dell'esperto i per il tempo t
 - 3: Il nostro algoritmo deve scommettere su 0 o su 1. Sia $z_t \in \{0, 1\}$ la previsione dell'algoritmo per il tempo t . L'unico modo per certificare la risposta è aspettare il tempo t e vedere cosa succede nella realtà $x_t \in \{0, 1\}$.
 - 4: Se $z_t \neq x_t$ paghiamo
 - 5: **end for**
-

Possiamo rappresentare il comportamento dell'algoritmo con una tabella:

Algorithm 6 del dimezzamento

- 1: $S \leftarrow \{1, 2, \dots, n\}$
 - 2: **for** $t = 1$ **to** T **do**
 - 3: Sia z_t la scelta di maggioranza degli esperti in S
 - 4: Osserviamo x_t
 - 5: $S \leftarrow S \setminus \{i \in S \mid y_{it} \neq x_t\}$
 - 6: **end for**
-

Teorema 4.1. *Se esiste un esperto perfetto (un esperto che non sbaglia mai), allora l'algoritmo del dimezzamento fa al più $\lceil \log_2 n \rceil$ errori (dato che scegliamo la maggioranza degli esperti). Se l'algoritmo sbaglia allora almeno la metà degli esperti sbaglia. Se l'algoritmo sceglie un esperto in maniera casuale, allora l'algoritmo commetterà al più n errori.*

	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$
E_1	0				
E_2	1	0			
E_3	1	1	0		
E_4	1	1	1	0	
E_5 (esperto perfetto)	1	1	1	1	0
A	0 (sceglie E_1)	0 (sceglie E_2)	0 (sceglie E_3)	0 (sceglie E_4)	0
R	1	1	1	1	0

Algorithm 7 Weighted Majority

```
1: function WEIGHTED-MAJORITY
2:   Si assegni un peso  $w_i = 1$  ad ogni esperto  $i \in [n]$ 
3:   for  $t = 1$  to  $T$  do
4:     Sia  $A_t \leftarrow \sum_{i|Y_{it}=1} w_i$  la somma dei pesi degli esperti che hanno predetto 1 al tempo  $t$ 
5:     e  $B_t \leftarrow \sum_{i|Y_{it}=0} w_i$  la somma dei pesi degli esperti che hanno predetto 0 al tempo  $t$ 
6:     if  $A_t \geq B_t$  then
7:        $z_t \leftarrow 1$ 
8:     else
9:        $z_t \leftarrow 0$ 
10:    end if
11:    Osserviamo  $x_t$ 
12:    for  $i = 1$  to  $n$  do
13:      if  $y_{it} \neq x_t$  then
14:         $w_i \leftarrow \frac{w_i}{2}$ 
15:      end if
16:    end for
17: end function
```

Teorema 4.2. *L'algoritmo del Weighted Majority farà un numero di errori che soddisfa la seguente disequazione:*

$$m \leq 2.41 \cdot (m^* + \log_2 n)$$

dove m^* è il numero di errori dell'esperto che ha sbagliato di meno.

Dimostrazione. Sia w_i^t il peso dell'esperto i alla fine del round t . Sia $w_i^0 = 1 \forall i \in [n]$. Il nostro potenziale è $W^t = \sum_{i=1}^n w_i^t$

A. $W^0 = \sum_{i=1}^n w_i^0 = n$

B. $W^t \geq W^{t+1} \forall t \geq 0$ (il potenziale decresce)

C. Se $z_t \neq x_t$ (ovvero il nostro algoritmo sbaglia al tempo t), allora $W^t \leq \frac{3}{4} W^{t-1}$.

Infatti, se $z_t \neq x_t$ allora all'inizio del round t il peso totale I^{t-1} degli esperti che sbagliano al round t soddisfa

$$I^{t-1} \geq \frac{W^{t-1}}{2}$$

allora

$$W^t = \frac{I^{t-1}}{2} + (W^{t-1} - I^{t-1}) = W^{t-1} - \frac{I^{t-1}}{2} \leq W^{t-1} - \frac{W^{t-1}}{4} = \frac{3}{4} W^{t-1}$$

Ovvero il potenziale decresce molto velocemente quando l'algoritmo sbaglia

D. Alla fine del round T (l'ultimo round), se Weighted Majority ha fatto m errori, allora:

$$W^T \leq \left(\frac{3}{4}\right)^m W^0 = \left(\frac{3}{4}\right)^m n$$

E. $W^T = \sum_{i=1}^n w_i^T \geq w_{i^*}^T$ dove i^* è l'esperto che ha fatto il minimo numero di errori m^*

F. $w_{i^*}^T = 2^{-m^*}$

Quindi

$$2^{-m^*} \leq w_{i^*}^T \leq W^T \leq \left(\frac{3}{4}\right)^m n$$

da cui

$$\begin{aligned}
\left(\frac{3}{4}\right)^m n &\geq 2^{-m^*} \\
m \log_2 \frac{3}{4} + \log_2 n &\geq -m^* \\
-m \log_2 \frac{4}{3} + \log_2 n &\geq -m^* \\
m^* + \log_2 n &\geq m \log_2 \frac{4}{3} \\
m &\leq \frac{1}{\log_2 \frac{4}{3}} (m^* + \log_2 n) \leq 2.41(m^* + \log_2 n)
\end{aligned}$$

□

Questo algoritmo è deterministico, tuttavia esistono algoritmi probabilistici che fanno meno errori.

4.1 Algoritmi non-random per il problema degli esperti

Sia A un algoritmo non-random generico. Consideriamo la seguente istanza:

	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = T$
E_1	0	0	0	0	0
E_2	1	1	1	1	1
Alg.	a_1	a_2	a_3		a_T
R	$1 - a_1$	$1 - a_2$	$1 - a_3$		$1 - a_T$

Lemma 4.3. *L'algoritmo A farà T errori in questa istanza*

Lemma 4.4. *Il migliore tra i due esperti farà $\leq \frac{T}{2}$ errori*

Teorema 4.5. *Nessun algoritmo non-random può garantire un numero di errori m minore di $m \leq 2m^*$ (dove m^* è il numero di errori del migliore esperto)*

Vediamo adesso un'algoritmo randomizzato per il problema degli esperti: Randomized Weighted Majority

Algorithm 8 Randomized Weighted Majority

```

1: function RWM $_{\varepsilon}$ 
2:    $w_i \leftarrow 1 \ \forall i \in [n]$ 
3:   for  $t = 1$  to  $T$  do
4:     Si consideri la distribuzione di probabilità  $P_t$  che assegna probabilità  $\frac{w_j}{\sum_{k=1}^n w_k}$  all'esperto
        $j$ 
5:     Campioniamo  $i$  da  $P_t$ 
6:     Prediciamo  $z_t = y_{it}$ 
7:     Osserviamo  $x_t$ 
8:     for  $i = 1$  to  $n$  do
9:       if  $y_{it} \neq x_t$  then
10:         $w_i \leftarrow (1 - \varepsilon)w_i$ 
11:       end if
12:     end for
13:   end for
14: end function

```

Mentre nel Weighted Majority l'algoritmo sceglieva la previsione della maggioranza degli esperti, in questo caso l'algoritmo sceglie una previsione con probabilità proporzionale al peso degli esperti.

Teorema 4.6. Sia fissato $0 < \varepsilon < \frac{1}{2}$. Il valore atteso del numero di errori compiuti dall'algoritmo soddisfa la seguente disuguaglianza:

$$\mathbb{E}[m] \leq (1 + \varepsilon)m^* + \frac{1}{\varepsilon} \ln n$$

dove m^* è il numero di errori compiuti dal migliore esperto.

Dimostrazione. Sia w_i^t il peso dell'esperto i alla fine del round t . Inoltre sia $w_i^0 = 1 \ \forall i \in [n]$ e sia $W^t = \sum_{i=1}^n w_i^t$. Sia I^t la frazione pesata degli esperti che sbagliano al tempo t :

$$I^t = \frac{\sum_{i=1}^n w_i^{t-1} \mathbb{1}_{y_{it} \neq x_t}}{\sum_{i=1}^n w_i^{t-1}} = \frac{\sum_{i=1}^n w_i^{t-1} \mathbb{1}_{y_{it} \neq x_t}}{W^{t-1}}$$

Supponiamo che i^* sia l'esperto migliore, che ha compiuto m^* errori nei primi T round. Allora

$$w_{i^*}^T = w_{i^*}^0 (1 - \varepsilon)^{m^*} = (1 - \varepsilon)^{m^*}$$

quindi $W^T \geq w_{i^*}^T = (1 - \varepsilon)^{m^*}$. Inoltre

$$\begin{aligned} \mathbb{E}[m] &= \sum_{t=1}^T P(\text{RWM}_\varepsilon \text{ fa un errore al round } t) = \\ &= \sum_{t=1}^T P(\text{RWM}_\varepsilon \text{ scelga al round } t \text{ un esperto che sbaglia}) = \\ &= \sum_{t=1}^T I^t \end{aligned}$$

Sia $t \geq 0$ e

$$\begin{aligned} W^{t+1} &= \sum_{i=1}^n w_i^{t+1} = \\ &= \sum_{i=1}^n (w_i^t (1 - \varepsilon \cdot \underbrace{\mathbb{1}_{y_{i,t+1} \neq x_{t+1}}}_{\text{indicatrice}})) = \\ &= \sum_{i=1}^n w_i^t - \varepsilon \sum_{i=1}^n w_i^t \cdot \mathbb{1}_{y_{i,t+1} \neq x_{t+1}} = \\ &= \sum_{i=1}^n w_i^t - \varepsilon \sum_{i=1 | y_{i,t+1} \neq x_{t+1}}^n w_i^t = \\ &= \sum_{i=1}^n w_i^t - \varepsilon W^t \cdot \sum_{i=1 | y_{i,t+1} \neq x_{t+1}}^n \frac{w_i^t}{W^t} = \\ &= \sum_{i=1}^n w_i^t - \varepsilon W^t \cdot I^{t+1} = \\ &= W^t - \varepsilon W^t \cdot I^{t+1} = \\ &= W^t (1 - \varepsilon I^{t+1}) \end{aligned}$$

Ma, allora,

$$\begin{aligned}
W^0 &= \sum_{i=1}^n w_i^0 = n \\
W^1 &= W^0(1 - \varepsilon I^1) = n(1 - \varepsilon I^1) \\
W^2 &= W^1(1 - \varepsilon I^2) = n(1 - \varepsilon I^1)(1 - \varepsilon I^2) \\
&\vdots \\
W^T &= n \prod_{s=1}^t (1 - \varepsilon I^s)
\end{aligned}$$

Infine

$$n \prod_{s=1}^t (1 - \varepsilon I^s) = W^t \geq (1 - \varepsilon)^{m^*} \Rightarrow \ln n + \sum_{s=1}^T \ln(1 - \varepsilon I^s) \geq m^* \ln(1 - \varepsilon)$$

ma vale anche:

$$\ln n + \sum_{s=1}^T \ln(1 - \varepsilon I^s) \leq \ln n + -\varepsilon \sum_{s=1}^T I^s$$

Dato che $\ln(1 - x) \leq -x \forall x \in (0, 1)$. Quindi

$$\begin{aligned}
\ln n &\geq m^* \ln(1 - \varepsilon) + \varepsilon \sum_{s=1}^T I^s \\
\ln n &\geq m^* \ln(1 - \varepsilon) + \varepsilon \mathbb{E}[m] \\
\ln n + m^* \ln \frac{1}{1 - \varepsilon} &\geq \varepsilon \mathbb{E}[m] \\
\mathbb{E}[m] &\leq \frac{1}{\varepsilon} \ln n + \frac{\ln \frac{1}{1 - \varepsilon}}{\varepsilon} m^* \\
&\leq \frac{1}{\varepsilon} \ln n + \frac{\varepsilon + \varepsilon^2}{\varepsilon} m^* \\
&= (1 + \varepsilon) m^* + \frac{1}{\varepsilon} \ln n
\end{aligned}$$

Dato che $\ln \frac{1}{1 - \varepsilon} \leq \varepsilon + \varepsilon^2 \forall \varepsilon \in (0, \frac{1}{2})$

□

4.2 Clustering

Raggruppare insiemi di dati in base a delle caratteristiche comuni. Dato un insieme di elementi e una funzione di somiglianza, l'obiettivo è quello di raggruppare gli elementi in modo che elementi dello stesso gruppo siano simili tra loro e elementi di gruppi diversi siano dissimili. La ragione per cui si fa clustering è che questo permette di trovare delle strutture nascoste nei dati.

Presentiamo la similarità di Jaccard:

Definizione 4.7 (Similarità di Jaccard). Dati due insiemi A e B , la similarità di Jaccard è definita come:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Definizione 4.8 (Distanza di Jaccard). La distanza di Jaccard è definita come:

$$d(A, B) = 1 - J(A, B)$$

Vediamo le proprietà di un buon clustering:

- **Invarianza di scala:** Se moltiplico tutti i dati per una costante, il clustering non cambia.

- **Consistenza:** Se avvicinano due punti che sono nello stesso cluster e allontanano due punti che sono in cluster diversi, il clustering non cambia.
- **Ricchezza:** un buon clustering deve essere sempre in grado di trovare un cluster se esiste un partizionamento naturale.

Osservazione 4.9. Non esiste un algoritmo che soddisfi tutte queste proprietà. L'algoritmo di single linkage è un esempio di algoritmo di clustering. Un punto fondamentale sono le condizioni della terminazione, per evitare di avere un solo cluster. Si dimostra che, in base alle condizioni di terminazione, si ha:

- Ricchezza e consistenza se si decide che tutte le distanze rimanenti sono maggiori di un certo valore fissato r
- Invarianza di scala e ricchezza se si decide che tutte le distanze rimanenti sono più grandi della distanza massima moltiplicata per $x \in (0, 1)$
- Invarianza di scala e consistenza se si decide si vuole un numero fisso k di cluster

Single Linkage ci restituisce un albero di clusters.

Definizione 4.10 (Locality Sensitive Hashing). LSH è una tecnica che permette di trovare elementi simili in un insieme di dati. L'idea è quella di mappare gli elementi in un insieme di hash tables in modo che elementi simili abbiano lo stesso hash. Un LSH per una funzione di similarità $S : U \times U \rightarrow [0, 1]$ è una distribuzione di probabilità su un insieme di funzioni di hash $\mathcal{H}$ tale che:

$$Pr_{h \in \mathcal{H}}[h(A) = h(B)] = S(A, B) \quad \forall A, B \in \mathcal{H}$$

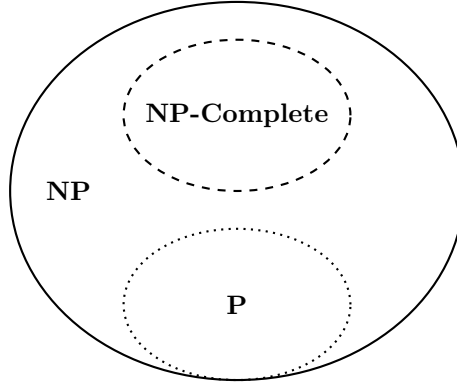
5 Fondamenti di complessità

Definizione 5.1 (Problemi NP). Un problema decisionale appartiene alla classe **NP** se, per ogni istanza in cui la risposta è "sì", esiste una certificazione (o prova) della validità di questa risposta che può essere verificata in tempo polinomiale da un algoritmo deterministico. Formalmente, un problema è in NP se una soluzione proposta può essere verificata in tempo polinomiale rispetto alla dimensione dell'input.

Definizione 5.2 (Problemi P). Un problema decisionale appartiene alla classe **P** se esiste un algoritmo deterministico che lo risolve in tempo polinomiale rispetto alla dimensione dell'input. In altre parole, per ogni istanza del problema, l'algoritmo fornisce una risposta ("sì" o "no") in un numero di passi che è una funzione polinomiale della dimensione dell'input.

Definizione 5.3 (Problemi NP-Completi). Un problema decisionale è definito **NP-completo** se soddisfa due condizioni:

1. **Appartenenza a NP:** Il problema è in NP, ovvero le soluzioni affermative possono essere verificate in tempo polinomiale.
2. **NP-difficoltà:** Ogni problema in NP può essere trasformato (ridotto) in tempo polinomiale in questo problema. Ciò implica che, se esistesse un algoritmo in grado di risolvere un problema NP-completo in tempo polinomiale, allora tutti i problemi in NP sarebbero risolvibili in tempo polinomiale.



Problema 5.1 (Satisfiability). Dato un insieme di variabili booleane $x_1, x_2, \dots, x_n$:

$$F(x_1, x_2, \dots, x_n) = \bigwedge_{i=1}^m C_i$$

dove C_i è una clausola, ovvero una disgiunzione di letterali:

$$C_i = \bigvee_{j=1}^k \lambda_{ij} \in \{x_1, x_2, \dots, x_n\} \cup \{\neg x_1, \neg x_2, \dots, \neg x_n\}$$

Esempio 5.4.

$$F(x_1, x_2, x_3) = \overbrace{(x_1 \vee x_2 \vee x_3)}^{C_1} \wedge \overbrace{(\neg x_1 \vee \neg x_4 \vee \neg x_5)}^{C_2} \wedge \dots \wedge \overbrace{(\neg x_2 \vee \neg x_4 \vee x_5)}^{C_n}$$

La domanda che ci poniamo è: esiste un assegnamento di verità alle variabili $x_1, x_2, \dots, x_n$ tale che $F(x_1, x_2, \dots, x_n) = T$ ovvero F è soddisfacibile? Formalmente lo possiamo scrivere come:

$$\exists \tau : X \rightarrow \{T, F\} \text{ t.c. } F(\tau(x_1), \tau(x_2), \dots, \tau(x_n)) = T$$

Teorema 5.5 (Clique). *Si da in input un grafo $G = (V, E)$ e un intero $k > 0$. La domanda è: esiste un sottoinsieme di k nodi di V che formano una clique? La clique massima di un grafo si indica con $\omega(G)$. Una clique è un sottoinsieme di nodi di un grafo tale che ogni coppia di nodi è connessa da un arco. Quindi dato $C = \{v_1, \dots, v_k\}$ questo insieme di nodi è una clique se $\forall v_i, v_j \in C$ esiste un arco $(v_i, v_j) \in E$.*

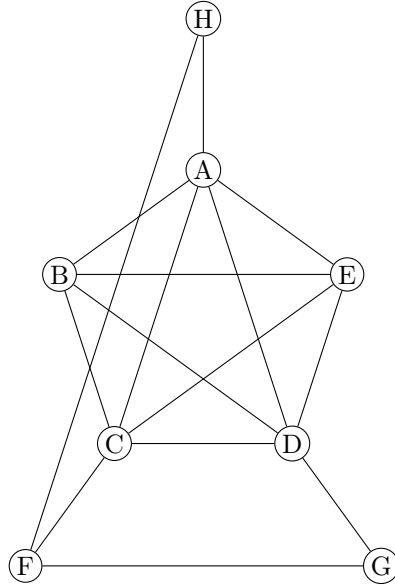


Figura 7: Esempio di clique di 5 nodi in un grafo

Se si suppone l'esistenza di un algoritmo efficiente che risolve il problema della clique:

$$A(G, k) = \begin{cases} 1 & \text{se } \omega(G) \geq k \\ 0 & \text{altrimenti} \end{cases}$$

Diciamo che il problema della soddisfacibilità (SAT) $\leq$ problema della clique. Ovvero esiste f che lavora in tempo polinomiale tale che:

$$F \xrightarrow{f} G, k \\ F \in \text{SAT} \Leftrightarrow \omega(G) \geq k \quad ((G, k) \in \text{Clique})$$

Si osservi che G, k non può essere in forma esponenziale, dato che, per definizione, f lavora

```

1: function SAT-SOLVER( $F$ )
2:    $G, k \leftarrow f(F)$ 
3:   if  $A(G, k) = 1$  then
4:     return True
5:   else
6:     return False
7:   end if
8: end function

```

in tempo polinomiale. Quindi, se esiste un algoritmo polinomiale per la clique, allora esiste un algoritmo polinomiale per la soddisfacibilità.

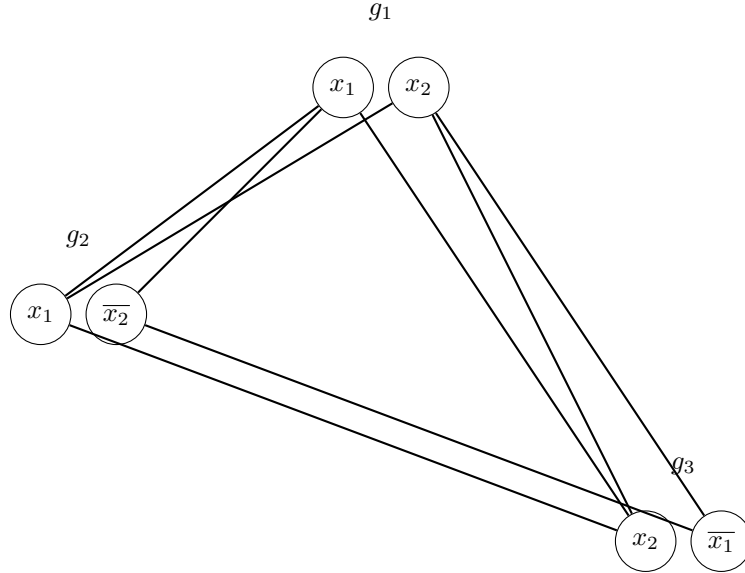
$$F \longrightarrow G, k = m \\ F(x_1, x_2, \dots, x_n) = C_1 \wedge C_2 \wedge \dots \wedge C_m \\ F(x_1, x_2) = \underset{\lambda_{11}\lambda_{12}}{(x_1x_2)} \underset{\lambda_{21}\lambda_{22}}{(x_1\neg x_2)} \underset{\lambda_{31}\lambda_{32}}{(\neg x_1x_2)}$$

Nel grafo che si costruisce, si mettono in relazione i letterali e i nodi:

$$V := \{ \overset{\lambda_{11}}{\circ} \overset{\lambda_{12}}{\circ} \dots \overset{\lambda_{mk}}{\circ} \}$$

Ci sono delle condizioni per cui due nodi sono connessi da un arco. $uv \in E$ se:

- u, v sono in gruppi diversi
- Non ci può essere un arco tra due affermazioni che sono una la negazione dell'altra



Questo si può calcolare in tempo polinomiale. Se $F \in \text{SAT}$ allora $\exists \tau : X \rightarrow \{T, F\}$ con $F(\tau(x_1), \tau(x_2), \dots, \tau(x_n)) = T$.

$$F = C_1 \wedge C_2 \wedge \dots \wedge C_m$$

$\lambda_1 \quad \lambda_2 \quad \lambda_m$

Dove ogni λ_i è il letterale vero. Posso quindi costruire una clique di dimensione m , dato che sono rispettate le condizioni per cui due nodi sono connessi da un arco.

Vediamo se è vero anche il contrario: sia $\omega(G) \geq k$ e la clique $C = \{u_1, u_2, \dots, u_m\}$: Ho quindi i letterali: $\lambda_1, \lambda_2, \dots, \lambda_m$.

$$\tau : \lambda_1 = T \quad \lambda_2 = T \quad \dots \quad \lambda_m = T$$

Per costruzione, τ è un assegnamento di verità che soddisfa F . Quella che abbiamo appena fatto è una riduzione polinomiale. Se esiste un algoritmo polinomiale per la clique, allora esiste un algoritmo polinomiale per la soddisfacibilità.

Definizione 5.6 (Problema universale). $X \in \text{NP}$ è NP-completo se $\forall Y \in \text{NP}, Y \leq X$. Ovvero, se esiste una riduzione polinomiale da Y a X .

Proposizione 5.7. SAT è NP-completo (Cook-Levin)

Clique è quindi NP-completo, dato che $\forall x \in \text{NP } X \leq \text{Clique}$.

$$X \stackrel{f}{\leq} \text{SAT} \stackrel{g}{\leq} \text{Clique}$$

Consideriamo ora il problema "Independent Set".

Problema 5.2 (Independent Set). Dato un grafo $G = (V, E)$ e un intero $k > 0$, la domanda è: esiste un sottoinsieme di k nodi di V che formano un insieme indipendente? Un insieme indipendente è un sottoinsieme di nodi di un grafo tale che nessuna coppia di nodi è connessa da un arco. Ovvero:

$$\exists I \subseteq V \quad \forall u, v \in I \quad (u, v) \notin E$$

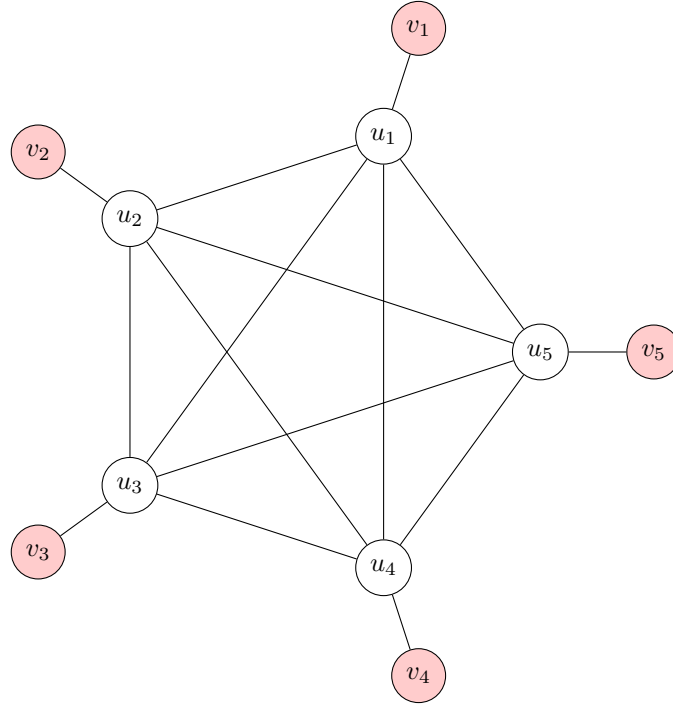


Figura 8: nodi in rosso costituiscono l'indendent set

Vogliamo capire se possiamo ridurre il problema della clique a quello dell'Independent Set. Ovvero: $\text{Clique} \leq \text{Independent Set}$. Dobbiamo trovare quindi una funzione polinomiale f tale che:

$$G, k \xrightarrow{f} G', k'$$

$$\langle G, k \rangle \in \text{Clique} \Leftrightarrow \langle G', k' \rangle \in \text{Independent Set}$$

Possiamo fare la riduzione in questo modo: $G' = G^c$ ovvero il complemento di G . $k' = k$.

$$E(G^c) = \{(u, v) \mid u, v \notin E(G)\}$$

Problema 5.3 (Vertex Cover). Dato un grafo $G = (V, E)$ e un intero $k > 0$, la domanda è: G contiene un cover di dimensione $\leq k$:

$$\exists C \subseteq V \quad \forall (u, v) \in E \quad u \in C \vee v \in C$$

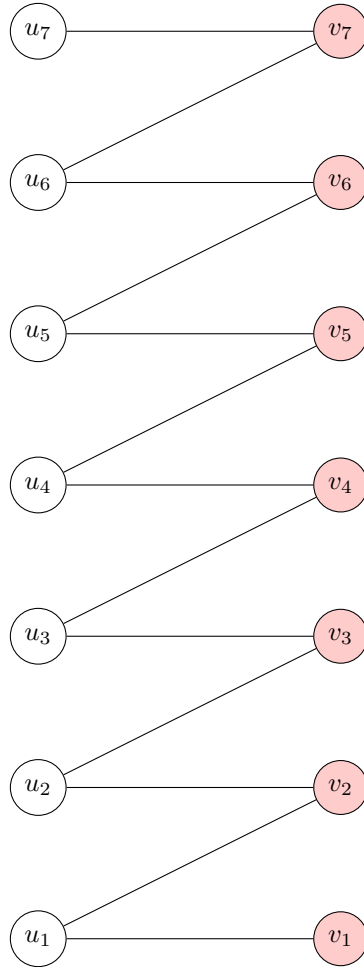


Figura 9: vertex cover in rosso

Comunque prendo un arco, almeno uno dei due nodi deve essere nel cover.

Vediamo se possiamo dimostrare $IS \leq VC$:

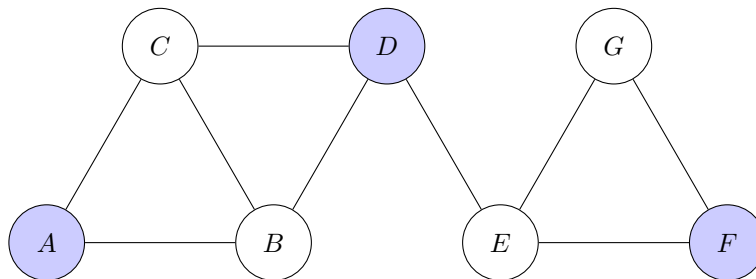
$$G, k \xrightarrow{f} G', k'$$

$$\langle G, k \rangle \in \text{Independent Set} \Leftrightarrow \langle G', k' \rangle \in \text{Vertex Cover}$$

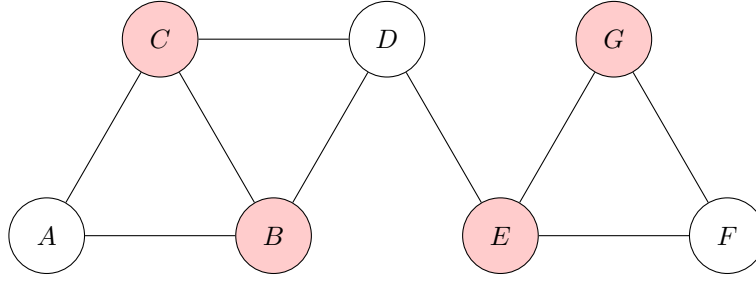
Possiamo osservare che I è un insieme indipendente se e solo se $V \setminus I$ è un cover (si dimostra). Quindi possiamo fare la riduzione in questo modo: $G' = G$, $k' = n - k$.

Vogliamo illustrare come un insieme indipendente di dimensione k in un grafo G corrisponda a un vertex cover di dimensione $k' = n - k$, dove n è il numero totale di nodi in G .

Per dare un'idea della dimostrazione illustriamo graficamente una possibile situazione.



Insieme indipendente ($k = 3$) e Vertex Cover corrispondente ($k' = n - k = 4$)



I nodi colorati in blu (A, D, F) formano un insieme indipendente di dimensione $k = 3$. Nodi colorati in rosso (B, C, E, G) formano un vertex cover di dimensione $k' = 4$.

Noi lavoreremo con insiemi finiti Σ (detto alfabeto) e $\Sigma^* := \{x_1x_2\dots x_n : k \geq 0, x_i \in \Sigma\}$ (detto insieme delle stringhe su Σ). Un sottoinsieme $A \subseteq \Sigma^*$ è detto *linguaggio*.

Definizione 5.8 (Problema decisionale). Un problema decisionale ha in input una stringa $x \in \Sigma^*$ e la domanda è: $x \in A$?

Ad esempio, per il problema della soddisfacibilità avevamo:

$$\begin{aligned} F(x_1, x_2, \dots, x_n) &= C_1 \wedge C_2 \wedge \dots \wedge C_m \\ C_i &= \lambda_{i1} \vee \lambda_{i2} \vee \dots \vee \lambda_{ik} \\ \lambda_{ij} &\in \{x_1, x_2, \dots, x_n\} \cup \{\neg x_1, \neg x_2, \dots, \neg x_n\} \end{aligned}$$

Per cui si ha l'alfabeto $\Sigma = \{T, F, \wedge, \vee, \neg, (,), =, _i\}$.

Per il problema della clique, dove abbiamo $G = (V, E)$ con $|V| = n$, $|E| = m$. Inoltre $V = [n] = \{1, 2, \dots, n\}$. Quindi possiamo costruire l'alfabeto $\Gamma = \{0, \dots, 9, V, E, =, (,), ', [,], k\}$.

Definizione 5.9. Dati il linguaggio $A \subseteq \Sigma^*$ e $B \subseteq \Gamma^*$, diciamo che $A \leq B$ se esiste una funzione calcolabile in tempo polinomiale (un algoritmo che si programma) $f : \Sigma^* \rightarrow \Gamma^*$ tale che:

$$x \in A \Leftrightarrow f(x) \in B$$

Osservazione 5.10. Se $A \leq B$ e $B \in P$ allora $A \in P$. Dove P è la classe dei problemi decisionali che possono essere risolti in tempo polinomiale.

Quindi, fino a questo punto, abbiamo visto che:

$$\text{SAT} \leq \text{Clique} \leq \text{IS} \leq \text{VC}$$

Teorema 5.11 (Sat è universale (NP-completo)). $\forall x \in NP \ x \leq \text{SAT}$

quindi se $\text{SAT} \leq Z$ allora $Z \in \text{NP-completo}$.

Problema 5.4 (3-SAT). Il problema della 3 soddisfacibilità è identico al problema SAT, con la differenza che ogni clausola C_i è una disgiunzione di al più 3 letterali: $|C_i| \leq 3$. Mostriamo che $\text{SAT} \leq 3\text{-SAT}$, ovvero che:

$$F \xrightarrow{f} F'$$

Dove F è un'istanza del problema SAT, F' è un'istanza del problema 3-SAT e f è una funzione calcolabile in tempo polinomiale.

Sia ϕ una formula in SAT arbitraria. Vogliamo costruire una formula ϕ' in 3-SAT tale che ϕ è soddisfacibile se e solo se ϕ' è soddisfacibile. La costruzione di ϕ' avverrà attraverso una trasformazione in tempo polinomiale.

Vediamo la costruzione di ϕ' :

La formula ϕ è composta da clausole $C_1, C_2, \dots, C_m$, dove ogni clausola C_i è una disgiunzione di letterali. Trasformeremo ciascuna clausola C_i in clausole di lunghezza ≤ 3 . Se una clausola C_i ha già lunghezza ≤ 3 , non è necessario alcun cambiamento. Sia $C_i = (l_1 \vee l_2 \vee \dots \vee l_k)$ con $k > 3$, introduciamo nuove variabili ausiliarie $y_1, y_2, \dots, y_{k-3}$ e sostituiamo C_i con le seguenti clausole:

$$(l_1 \vee l_2 \vee y_1), (\neg y_1 \vee l_3 \vee y_2), (\neg y_2 \vee l_4 \vee y_3), \dots (\neg y_{k-4} \vee l_{k-2} \vee y_{k-3}), (\neg y_{k-3} \vee l_{k-1} \vee l_k).$$

Questa trasformazione garantisce che ogni clausola ha esattamente 3 letterali.

Vediamo quindi la correttezza della trasformazione:

Dobbiamo mostrare che ϕ è soddisfacibile se e solo se ϕ' è soddisfacibile.

Se ϕ è soddisfacibile, allora ϕ' è soddisfacibile

Supponiamo che esista un'assegnazione soddisfacente A per ϕ . Dobbiamo estendere A per assegnare valori alle nuove variabili ausiliarie in ϕ' in modo che tutte le clausole siano soddisfatte.

Per ogni clausola trasformata come sopra, possiamo assegnare i valori alle variabili ausiliarie in modo sequenziale:

- Se l_1 o l_2 è vero sotto A , assegnare y_1 arbitrariamente (ad esempio, vero).
- Se l_1 e l_2 sono falsi, ma l_3 è vero, assegnare $y_1 = \text{vero}$ per soddisfare la seconda clausola, e così via.

Se ϕ' è soddisfacibile, allora ϕ è soddisfacibile

Supponiamo che esista un'assegnazione soddisfacente A' per ϕ' . Poiché le variabili ausiliarie sono nuove, possiamo considerare l'assegnazione ristretta alle variabili originali di ϕ . Dobbiamo mostrare che questa assegnazione soddisfa ϕ .

Nelle clausole trasformate, per ogni clausola originale C_i , la soddisfacibilità delle clausole derivate implica che almeno uno dei letterali originali l_j è vero sotto A' . Quindi, ogni clausola C_i è soddisfatta.

Pertanto, abbiamo che $\text{SAT} \leq 3\text{-SAT}$.

Problema 5.5 (3-Coloring). Dato un grafo $G = (V, E)$, la domanda è: esiste $x : V \rightarrow \{B, R, G\}$ tale che $\forall (u, v) \in E \ x(u) \neq x(v)$?

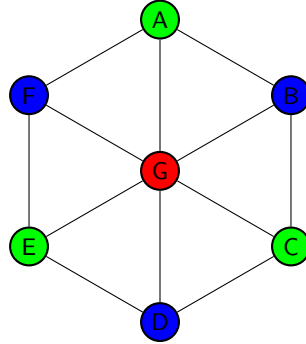


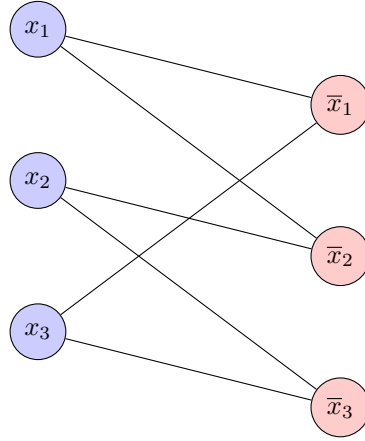
Figura 10: Esempio di grafo 3-colorabile

Vogliamo far vedere che $\text{SAT} \leq 3\text{-Coloring}$, e quindi che esiste f calcolabile in tempo polinomiale:

$$F \xrightarrow{f} G$$

$$F \in \text{SAT} \leftrightarrow G \in 3\text{-COL}$$

Osserviamo che la 2-colorabilità è verificata se e solo se il grafo è bipartito.



Per la 3 colorabilità possiamo costruire dei blocchi di grafi (gadget) che corrispondono a una clausola. Ad esempio, per la clausola i -esima $C_i = (x_1 \bar{x}_2 x_3 \bar{x}_4 x_5)$ corrisponde il gadget g_i . Per costruire il gadget facciamo riferimento al seguente grafo banale della tre colorabilità:

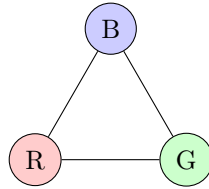


Figura 11: Triangolo 3-colorabile

Il gadget per la clausola C_i è quindi costruito nel modo seguente:

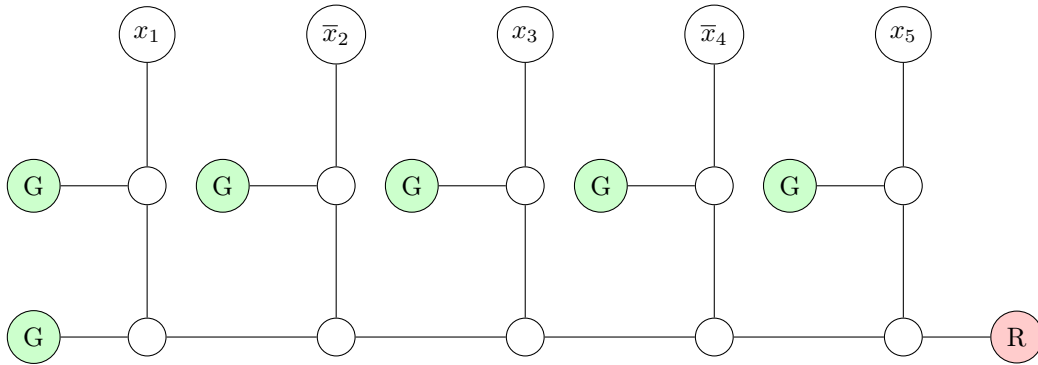
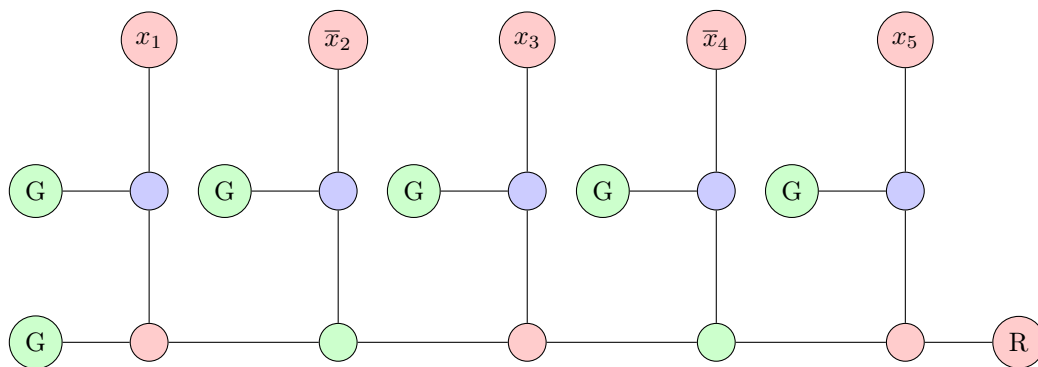


Figura 12: Gadget per la clausola C_i (lunghezza dispari)

Dove R e G sono tutti il vertice rosso e verde dello **stesso** triangolo in figura 11.

Osservazione 5.12. Se la clausola fosse stata di lunghezza **pari**, al posto del vertice rosso (R) **avremmo avuto un vertice verde (G)**.

Osservazione 5.13. Notiamo che se tutti i vertici fossero falsi (e quindi la clausola non vera) sarebbe impossibile completare la tre colorazione:

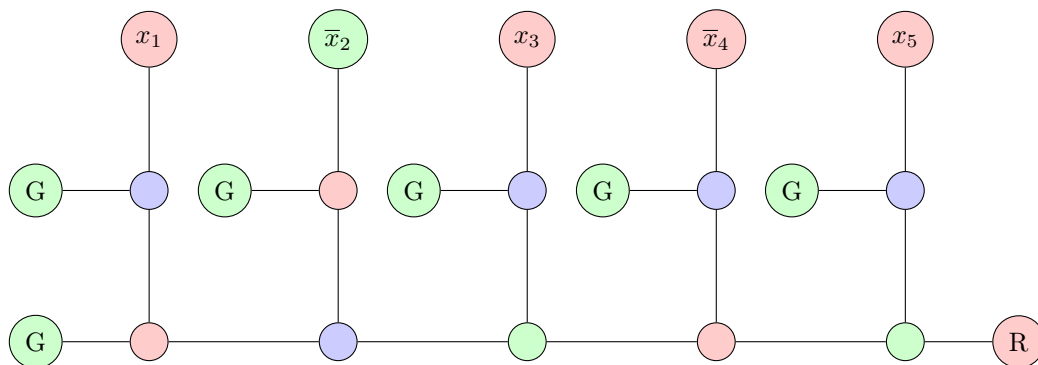


Mentre se almeno uno è vero allora possiamo completare la tre colorazione

Vediamo ora come $F \in \text{SAT}$ ci permette di costruire un grafo 3-colorabile: $\exists \tau : \{x_1 \dots x_n\} \rightarrow \{V, F\}$ e quindi:

$$F(\tau(x_1) \dots \tau(x_n)) = T$$

A questo punto abbiamo i colori di tutti i vertici e sappiamo che almeno uno è vero (per la soddisfacibilità). Quindi possiamo completare la colorazione.



Ora supponiamo G 3-colorabile, per cui esiste $x : V \rightarrow \{B, G, R\}$ e $\forall (u, v) \in E$ si ha $x(u) \neq x(v)$. Si arriva quindi a costruire $\tau : \{x_1 \dots x_n\} \rightarrow \{T, F\}$ e τ soddisfa F . Le dimostrazioni si basano quindi tutte sulla struttura del gadget. Questo perché se il grafo è 3-colorabile, allora ogni gadget corrispondente a una clausola è 3-colorabile e quindi almeno un vertice è colorato con un colore diverso dagli altri (vedi osservazione 5.13).

Problema 5.6 (Subset Sum). Dato $V = \{v_1, \dots, v_n\}$ e $t \in \mathbb{N}$ l'obiettivo è capire se $\exists A \subseteq V$ t.c. $\sum_{a \in A} a = t$

Dimostriamo che Subset Sum è NP: se ho una soluzione posso verificarla in tempo polinomiale attraverso un verificatore che prende in input (V, t) e la soluzione A e verifica che la somma degli elementi di A sia uguale a t .

$$\text{verificatore}((V, t), A) = \begin{cases} \text{Vero se } \sum_{a \in A} a = t \\ \text{Falso altrimenti} \end{cases}$$

Problema 5.7 (Exact cover). Dato (X, S) dove $X = \{0, 1, \dots, n-1\}$ e $S = \{S_1, S_2, \dots, S_m\}$ $S_i \subseteq X \forall i \in [m]$. L'obiettivo è capire se $\exists S' \subseteq S$ t.c. $\forall A, B \in S' A \neq B A \cap B = \emptyset$ e $\bigcup_{A \in S'} A = X$

Presupponiamo innanzitutto che $3\text{-SAT} \leq \text{Exact Cover}$. Ora vediamo se possiamo ridurre Exact Cover a Subset Sum. L'idea è quella di associare a ogni sottoinsieme di Exact Cover un input di

Subset Sum. Vediamo un esempio di come si può fare, ponendo $X = \{0, 1, 2, 3, 4\}$

$$\begin{aligned} S_1 = \{0, 2, 4\} &\longrightarrow v_1 = \begin{matrix} & 4 & 3 & 2 & 1 & 0 \\ (1 & 0 & 1 & 0 & 1) \end{matrix}_{m+1} \\ S_2 = \{1, 3\} &\longrightarrow v_2 = (0 \ 1 \ 0 \ 1 \ 0)_{m+1} \\ S_3 = \{0, 1, 3\} &\longrightarrow v_3 = \begin{matrix} (0 & 1 & 0 & 1 & 1) \\ =|X| & & & & m+1 \end{matrix} \\ &\vdots \end{aligned}$$

Usando la base $m+1$ riusciamo a verificare se la somma ci risulta in un'intersezione di insiemi che non hanno elementi in comune. Formalmente si scrive:

$$\forall i \in [m] \quad v_i = \sum_{a \in S_i} (m+1)^a$$

Per cui

$$t = \sum_{i=0}^{n-1} (m+1)^i = \frac{(m+1)^n - 1}{m}$$

Dimostriamo adesso che se Exact cover è risolvibile allora Subset sum è risolvibile. Infatti se Exact cover è risolvibile allora esiste $S' \subseteq S$ tale che:

1. $\forall A, B \in S' \quad A \neq B \text{ e } A \cap B = \emptyset$
2. $\bigcup_{A \in S'} A = X$

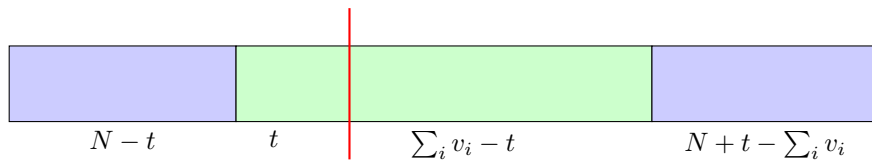
Per cui $A = \{v_i \mid S_i \in S'\}$. Dimostriamo adesso l'implicazione inversa: $A \subseteq V \quad \sum_{a \in A} a = t$. Possiamo considerare infatti:

$$S' = \{S_i \mid v_i \in A\}$$

Problema 5.8 (Partition). Il problema prende in input un insieme di n elementi $X = \{x_1, x_2, \dots, x_n\}$. L'obiettivo è trovare $Y \subseteq X$ tale che:

$$\sum_{y \in Y} y = \sum_{y \in X \setminus Y} y = \frac{1}{2} \sum_{x \in X} x$$

Il problema sembra molto simili a Subset Sum, vediamo quindi se $\text{Subset Sum} \leq \text{Partition}$. La differenza dell'input sta nel fatto che in Subset Sum c'è un target, mentre in partition no.



Possiamo quindi considerare

$$(V, t) \leftrightarrow X$$

Dove $X = V \cup \{N-t, N+t - \overbrace{\sum_i v_i}^\alpha\}$. Dimostriamo ora che la risoluzione di Subset Sum implica la risoluzione di Partition:

$$A \subseteq V \quad \sum_{a \in A} a = t$$

e posso vedere se $\sum_{a \in A \cup \{N-t\}} a = N$. Dimostriamo ora l'implicazione inversa. Data $Y \subseteq X$ dove $\sum_{y \in Y} y = N$. Per cui:

$$\Rightarrow N-t \in Y \wedge \alpha \notin Y \quad N-t \in X \setminus Y \wedge \alpha \in Y$$

e posso prendere $A = Y \setminus \{N-t\}$

Problema 5.9 (Knapsack). È dato in input una serie di oggetti con peso $w_1, w_2, \dots, w_n$ dove ogni oggetto ha un benefit $b_1, b_2, \dots, b_n$. È data ha una capacità W e un target B . L'obiettivo è dimostrare che esiste $A \subseteq [n]$ tale che:

1. $\sum_{i \in A} w_i \leq W$
2. $\sum_{i \in A} b_i \geq B$

Dimostriamo che $\text{Partition} \leq \text{Knapsack}$.

$$X \longrightarrow (\vec{w}, \vec{b}, W, B)$$

Possiamo farlo ponendo $w_i = x_i$, $b_i = x_i \quad \forall i$ e $W = B = \frac{1}{2} \sum_{i=1}^n x_i$. Vediamo come la risoluzione di Partition implica la risoluzione di Knapsack:

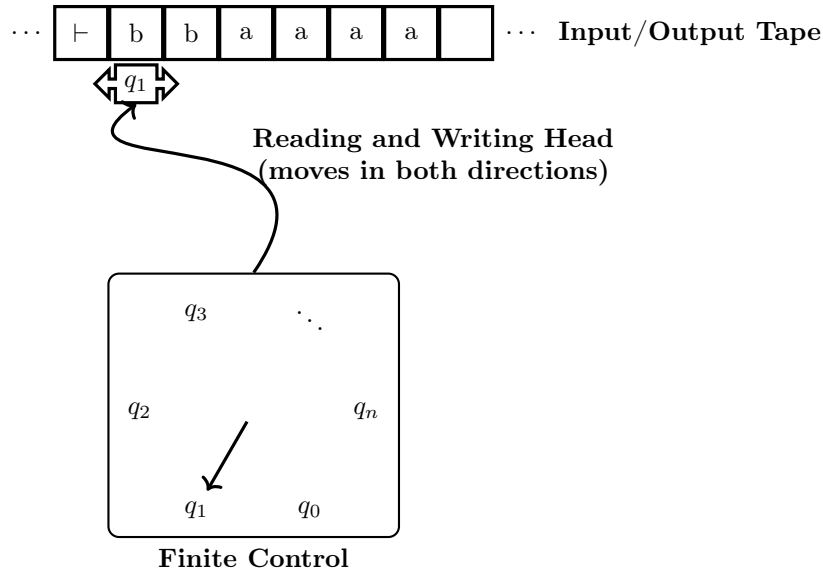
$$A \subseteq [n] \quad \sum_{i \in A} x_i = V = \underbrace{\sum_{i \in A} w_i}_{\leq W} = \underbrace{\sum_{i \in A} b_i}_{\geq B}$$

E, viceversa, la risoluzione di Knapsack in Partition pone:

$$A \subseteq [n] \quad \underbrace{\sum_{i \in A} w_i}_{\leq W} = V = \underbrace{\sum_{i \in A} b_i}_{\geq B} = \sum_{i \in A} x_i$$

6 Macchina di Turing

Nel 1936 Alan Turing pubblica un articolo in cui introduce la macchina di Turing. Questa macchina è un modello astratto di calcolatore: questo costituisce uno dei pilastri della teoria della computazione e della matematica del 21° secolo. Hilbert scrive “Das Entscheidungsproblem” (problema della decisione). Il problema è il seguente: dato un insieme di assiomi e un insieme di regole di inferenza, esiste un algoritmo che, dato un enunciato, stabilisce se è dimostrabile o meno?. Ecco una rappresentazione della macchina di Turing:



Quindi si ha una tabella di istruzioni finite del tipo:

$$\begin{aligned} (q, a) &\mapsto (q', a', m) \\ (p, b) &\mapsto (p', b', n) \\ &\vdots \end{aligned}$$

Dove $m, m' \in \{L, -, R\}$. Ogni processo di calcolo eseguibile da un computer può essere eseguito da una macchina di Turing (MT). Un'evidenza che la MT è un modello di calcolatore è il fatto che **ogni** linguaggio di programmazione moderno è equivalente alle TM's: qualunque processo di calcolo producibile dalla mente umana è programmabile (è un teorema). La MT è definita da una 8-tupla:

$$M = (\overbrace{Q, \Gamma, \Sigma}^{\text{finiti}}, \delta, \vdash, \sqcup, \underbrace{S, a, r}_{\in Q})$$

Q = insieme finito di stati
 Γ = alfabeto del nastro
 Σ = alfabeto di input $\Sigma \subsetneq \Gamma$
 $\delta : (Q \times \Gamma) \rightarrow (Q \times \Gamma \times \{L, R\})$
 $\vdash \in \Gamma - \Sigma$
 $\sqcup \in \Gamma - \Sigma$

Esempio 6.1. Supponiamo $\hat{\Sigma} = \{0, 1\}$ e $Q = \{1, 2, \dots, k_1\}$ $\Gamma = \{1, 2, \dots, k_2\}$ $\Sigma = \{1, 2, \dots, k_2\}$. Impostiamo la codifica della macchina di Turing nel seguente modo:

$$\langle M \rangle = 1 \underbrace{00\dots 0}_{|Q|} 1 \underbrace{0\dots 0}_{|\Gamma|} 1 \underbrace{0\dots 0 1}_{|\Sigma|}$$

e l'istruzione data da :

$$(q, a) \mapsto (q', a', m) \text{ con } m \in \{L, -, R\}$$

$q \in Q$ e $a \in \Gamma$. Ad esempio:

$$1 \underbrace{0\dots 0}_q 1 \underbrace{0\dots 0}_a 1 \underbrace{0\dots 0}_{q'} 1 \underbrace{0\dots 0}_{a'} 1 \underbrace{0\dots 0 1}_m$$

Un esercizio di programmazione può essere che dato in input una $\langle M \rangle, x$ si chiede di simulare $M(x)$. Un calcolatore moderno è una macchina di turing universale che simula degli algoritmi.

Modello della macchina di Turing è utile anche per studiare cose che gli algoritmi NON possono fare. Macchina di Turing è effettivamente un algoritmo, costruita mediante una "descrizione" (una codifica di zeri e di uno). La TM è deterministica.

Il problema dell'Halting prende in input una macchina di turing con un'istruzione x (ho in input un programma e l'input di quel programma), e chiede di verificare se quell'algoritmo entra in un ciclo infinito o meno. Turing dimostra che non esiste un algoritmo che risolve questo problema. Esplicitiamo la seguente funzione:

$$\hat{H}(i, j) = \begin{cases} \text{Yes se } M_i(j) \text{ si ferma} \\ \text{No se } M_i(j) \text{ non si ferma} \end{cases}$$

Possiamo rappresentare il comportamento dell'algoritmo con una tabella:

	1	2	3	...	i	...	$k \dots$	i	...
M_1	Y	N	...	Y	...	N	...	Y	...
M_2	N	Y	...	N	...	Y	...	N	...
$\vdots$	...	...	...	...	...	...	...	...	...
M_i	Y	...	N	...	Y	...	N	...	Y
$\vdots$	...	...	...	...	...	...	...	...	...
M_k	N	Y	...	N	...	Y	...	N	...
$\vdots$	...	...	...	...	...	...	...	...	...
M_j	Y	...	N	...	Y	...	N	...	Y
$\vdots$	...	...	...	...	...	...	...	...	...

Definiamo ora la funzione

$$H(i) \equiv \hat{H}(i, i) = \begin{cases} \text{Yes se } M_i(i) \text{ si ferma} \\ \text{No se } M_i(i) \text{ non si ferma} \end{cases}$$

Supponiamo per assurdo che H sia calcolabile, allora esiste un algoritmo (e quindi una TM) che la calcola, tale che $H(i) = M(i)$. Ma allora esiste anche D calcolabile:

$$D(x) := \begin{cases} \text{Yes se } H(x) = \text{No} \\ \text{No se } H(x) = \text{Yes} \end{cases}$$

Quindi D è una macchina di turing: senza perdita di generalità diciamo che si tratta della macchina M_k . Quindi si configurando due casi:

$$M_k(k) \text{ si ferma} \quad \text{oppure} \quad M_k(k) \text{ non si ferma}$$

Supponiamo quindi che $M_k(k)$ si ferma, allora $D(k) = \text{No}$. Se $M_k(k)$ non si ferma, allora $D(k) = \text{Yes}$. Quindi abbiamo che:

$$M_k(k) \neq D_k(k) \Rightarrow M_k \neq D_k$$

per cui H non può essere calcolabile.

Problema 6.1 (Decision problem). Il problema di decisione considera un sottoinsieme L di un alfabeto Σ^* e prende in input un alfabeto $x \in \Sigma^*$, per cui si chiede se $x \in L$ o meno.

Definizione 6.2 (Decidibile). Un linguaggio $L \subseteq \Sigma^*$ è decidibile se esiste una macchina di Turing M tale che $\forall x \in \Sigma^*$:

$$M(x) = \begin{cases} \text{Yes se } x \in L \\ \text{No se } x \notin L \end{cases}$$

Osservazione 6.3. H non è decidibile, così come tutte le domande sul “comportamento” dei programmi.

Esempio 6.4. Consideriamo il seguente linguaggio:

$$A = \{M\#x \mid M(x) \text{ accetta l'input}\}$$

Definizione 6.5. Dati $A \subseteq \Sigma^*$ e $B \subseteq \Gamma^*$ con $A \leq B$ esiste una funzione calcolabile $r : \Sigma^* \rightarrow \Gamma^*$ tale che:

$$x \in A \rightarrow r(x) \in B \quad x \notin A \rightarrow r(x) \notin B$$

Quindi è un fatto che se A è riducibile a B e B è decidibile, allora A è decidibile.

Corollario 6.6. Se $A \leq B$ e A non è decidibile, allora B non è decidibile.

Esempio 6.7. Vediamo la seguente riduzione: $H \leq A$ con $H \subseteq \Sigma^*$ e $A \subseteq \Gamma^*$. Per cui occorre trovare r calcolabile tale che:

$$M\#x \rightarrow N\#x$$

Definiamo quindi $N(z)$ che accetta z come input se $M(x)$ non si ferma. Abbiamo definito $H = \{M\#x \mid M(x) \text{ si ferma}\}$ e $A = \{M\#x \mid M(x) \text{ accetta}\}$.

1. Se $M\#x \in H$ allora $M(x)$ si ferma, per cui $N(x)$ accetta ogni input, di conseguenza $N\#x \in A$
2. Se $M\#x \notin H$ allora $M(x)$ non si ferma, per cui $N(x)$ non accetta nessun input, di conseguenza $N\#x \notin A$

7 Problemi indecidibili e non

Abbiamo visto i seguenti elementi che caratterizzano una macchina di Turing:

- Σ un alfabeto e $\Sigma \in \mathbb{N}$
- Σ^* l'insieme delle stringhe generabili da Σ : $\Sigma^* = \{x_1 x_2 \dots x_n \mid i \in \mathbb{N} \wedge \forall j x_j \in \Sigma\}$
- Definiamo un linguaggio un insieme $L \subseteq \Sigma^*$. Un linguaggio L è decidibile se esiste una macchina di Turing M tale che $\forall x \in \Sigma^*$ valgono 2 proprietà:
 1. $x \in L \rightarrow M(x) = \text{accetta}$
 2. $x \notin L \rightarrow M(x) = \text{rifiuta}$

Quindi un linguaggio è decidibile se esiste una macchina di Turing che accetta o rifiuta ogni stringa $x \in \Sigma^*$.

Teorema 7.1. *Il linguaggio $HP = \{M\#x \mid M(x) \text{ si ferma}\}$ non è decidibile. Tuttavia si dice che HP è semi-decidibile, dato che se $M(x)$ si ferma, allora $M(x)$ accetta, altrimenti non si può avere una risposta certa.*

Definizione 7.2. Una funzione $f : \Sigma^* \rightarrow \Sigma^*$ è computabile se esiste una macchina di Turing M tale che:

$$M(x) = f(x) \quad \forall x \in \Sigma^*$$

Definizione 7.3. Siano $A, B \subseteq \Sigma^*$ (dei linguaggi). Diciamo che $A \leq_T B$ se esiste una funzione calcolabile $r : \Sigma^* \rightarrow \Sigma^*$ tale che $\forall x \in \Sigma^*$ valgono:

1. $x \in A \rightarrow r(x) \in B$
2. $x \notin A \rightarrow r(x) \notin B$

Diciamo che r è detta “riduzione” da A a B .

Siano A, B due linguaggi. Se $A \leq_T B$ e B è decidibile, allora A è decidibile.

Corollario 7.4. *Siano A, B due linguaggi. Se $A \leq_T B$ e A non è decidibile, allora B non è decidibile.*

Esempio 7.5. $A = \{M\#x \mid \text{se } M(x) \text{ accetta}\}$. È sufficiente dare una riduzione che provi che $HP \leq_T A$. Possiamo definire $M\#x \rightarrow N\#x$ dove $N(z) = \text{“se } M(x) \text{ si ferma, allora accetta } z\text{”}$.

1. Se $M\#x \in HP$ allora $M(x)$ si ferma, per cui N accetta tutti gli input, quindi $N\#x \in A$
2. Se $M\#x \notin HP$ allora $M(x)$ non si ferma, per cui N non accetta nessun input, quindi $N\#x \notin A$

Per cui abbiamo dimostrato che $HP \leq_T A$, quindi A è indecidibile.

Esempio 7.6. Definiamo ora i linguaggi $E = \{M \mid M \text{ non accetta alcun input}\}$ (insieme delle macchine di turing che non accettano alcun input) e $F = \{M \mid M \text{ accetta almeno un input}\}$. Quindi F è il complemento di E . Possiamo quindi definire la seguente riduzione:

$$\begin{aligned} M\#x &\rightarrow N\#x \\ N(z) &: \text{se } M(x) \text{ accetta allora accetta } z \end{aligned}$$

Per cui $HP \leq_T F \leq_T E$. Quindi E e F sono indecidibili.

Esempio 7.7. Definiamo il linguaggio

$$EQ = \left\{ M_1\#M_2 \mid \forall x \in \Sigma^*, \begin{cases} \text{se } M_1(x) \text{ accetta allora } M_2(x) \text{ accetta} \\ \text{se } M_1(x) \text{ rifiuta allora } M_2(x) \text{ rifiuta} \\ \text{se } M_1(x) \text{ non si ferma allora } M_2(x) \text{ non si ferma} \end{cases} \right\} =$$

$$= \{M_1 \# M_2 \mid M_1 \text{ accetta allora } M_2 \text{ accetta e viceversa}\}$$

dimostriamo che EQ è indecidibile, dimostrando che $E \leq_T EQ$. Definiamo la seguente riduzione:

$$\begin{aligned} M &\longrightarrow M \# N \\ N(z) &: \text{ rifiuta } z \text{ a prescindere} \end{aligned}$$

Quindi $HP \leq_T F \leq_T E \leq_T EQ$. Quindi EQ è indecidibile.

I linguaggi decidibili sono numerabili, dato che per ogni linguaggio esiste una macchina di Turing che lo decide (ognuna che ha un numero finito di bit). I linguaggi non decidibili sono non numerabili. La macchina di Turing si ferma se non esiste una transizione possibile. La TM deterministica si accetta se si ferma su uno stato di accettazione $\square$. La TM deterministica non accetta se si ferma su uno stato non di accettazione $\circ$.

Le TM non deterministiche (estensione delle TM) detengono una computazione che può avere più di un ramo per ogni stato (per la stessa lettera in un determinato stato posso eseguire più transizioni). In altre parole, dallo stato generico leggendo il carattere generico, posso fare più di un'azione. La TM non deterministica accetta una parola se esiste un suo thread che si ferma su uno stato di accettazione. La TM N-det rifiuta se **ogni** suo thread si ferma su un qualche stato di non-accettazione.

Teorema 7.8. *L'insieme dei linguaggi (problemi) che possono essere decisi da TM deterministiche è identico all'insieme dei linguaggi decidibili da TM non deterministiche.*